

Vrije Universiteit Amsterdam



Universiteit van Amsterdam



Master Thesis

Found in translation: a LLM approach to private and reproducible Workflow Management Systems

Author: Emanuele Simeone (student number)

<i>1st supervisor:</i>	VU supervisor name
<i>daily supervisor:</i>	supervisor name
<i>2nd reader:</i>	supervisor name

*A thesis submitted in fulfillment of the requirements for
the joint VU-UvA Master of Science degree in Computer Science*

August 14, 2026

DECLARATION OF AI USAGE

I hereby acknowledge that the research reported in this thesis has been carried out with the responsible and ethical utilization of artificial intelligence (AI) technologies. AI tools such as ChatGPT- 4 and Gemini were utilized to assist in specific tasks, including:

- **Language and Grammar Assistance:** AI-based writing assistants were used to enhance the clarity and coherence of the text, ensuring proper grammar and style.
- **Learning:** AI-powered resources were instrumental in my learning process, helping me to clarify complex concepts, test my understanding through interactive questioning, and explore related research areas.

However, it is important to emphasize that the core ideas, research design, results, interpretations, and conclusions presented in this thesis are solely the product of my own intellectual effort. AI served as a supportive tool, not a substitute for critical thinking and independent research.

I acknowledge the limitations of AI and the potential for biases within its algorithms. Therefore, I have carefully reviewed and validated all AI-generated outputs to ensure their accuracy and alignment with the research objectives.

I accept complete accountability for the material presented in this thesis and assert that it complies with the utmost standards of academic integrity and ethical research practices.

ACKNOWLEDGMENTS

Please note that it is not appropriate to make any kind of personal (e.g., political or religious) statement in a thesis, nor in acknowledgments.

Here my acknowledgments...

NN

Place, Date

Abstract

Context. [at the end](#)

Goal. [at the end](#)

Method. [at the end](#)

Results. [at the end](#)

Conclusions. [at the end](#)

Contents

DECLARATION OF AI USAGE	ii
ACKNOWLEDGMENTS	iii
List of Figures	iv
List of Tables	vi
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.2.1 Scope of This Thesis	4
1.3 Research Questions	4
1.4 Thesis Structure	4
2 Background	6
2.1 Scientific Workflow Management Systems	6
2.1.1 Representative Systems	6
2.1.2 Workflow Lifecycle	7
2.2 Data Provenance	8
2.2.1 W3C PROV Data Model	8
2.2.2 Prospective vs. Retrospective Provenance	9
2.2.3 Workflow Provenance Formats	10
2.3 FAIR Principles for Workflows	10
2.4 Intent-Based Orchestration	11
2.4.1 Application to Scientific Computing	11
2.4.2 Gaps in the Literature	11
2.5 Large Language Models in Scientific Computing	12
2.5.1 LLMs for Code and Workflow Generation	12

CONTENTS

2.5.2	Retrieval-Augmented Generation	13
2.5.3	ControlA and Agentic Orchestration	13
2.6	Target Platform Selection: Brane and Dynamos	13
2.6.1	Brane and BraneScript	13
2.6.2	Alternative Considered: Dynamos	15
3	Overview	17
3.1	User Perspective	17
3.2	Process Architecture	18
3.3	Infrastructure Architecture	21
3.4	Interaction Sequence	23
3.5	Dashboard: Monitoring and Catalog Browsing	25
4	Design	28
4.1	Intent Decomposition	28
4.2	Retrieval Design	29
4.2.1	Chunking Granularity	30
4.2.2	Package Aliasing and Hybrid Retrieval	31
4.3	Prompt and Feedback Design	31
4.3.1	Failure Modes and Corrective Feedback	32
4.4	Typed Data Representation	32
4.5	Dataset Design	33
4.6	Fine-Tuning Design	34
4.7	Intent Provenance	36
4.8	Semantic Cache Design	38
5	Implementation	41
5.1	Pipeline Orchestrator	41
5.2	Knowledge Base and Retrieval	42
5.3	Brane Packages	42
5.4	Fine-Tuning Infrastructure	43
5.4.1	Training and Evaluation Scripts	43
5.5	Semantic Cache	44
5.5.1	Cache Evaluation Harness	44
5.6	Job Watcher	45
5.7	Dashboard	45

CONTENTS

5.8 Infrastructure	46
References	47

List of Figures

2.1	The core PROV-DM graph: entities (yellow ellipses), activities (blue rectangles), and agents (orange pentagons) connected by the used , wasGeneratedBy , wasAssociatedWith , and wasAttributedTo relations, following standard PROV-O visual notation.	9
3.1	Conceptual architecture shared by both entry points. A cache hit returns a previous (intent, workflow, result) triple immediately; on a miss, the intent is first decomposed into a short list of sub-tasks, which sharpen the query passed to the <i>only</i> retrieval component in the system, PkgRetriever , fetching package and dataset context from a single ChromaDB store (brane_pkg_db); the BraneScript syntax reference is not retrieved but injected in full, statically, into every prompt. Brane’s own compiler and runtime, invoked as a single brane workflow run command, are the sole authority on whether the generated script is valid; a compilation or runtime failure feeds back into prompt construction as corrective context for a bounded number of retries, and a successful result is written to the semantic cache.	20
3.2	Physical deployment of the components shown in Figure 3.1, for the interactive (dashboard) entry point; discussed in the text above. Site A and Site B are shown as two independent examples of federated sites reachable from the same Brane control node.	22
3.3	Sequence of interactions for the interactive (dashboard) entry point, discussed in the text above.	24
3.4	The Runs page, showing aggregate statistics and a searchable, sortable history of every intent previously submitted through the dashboard.	26
3.5	The Packages catalog, listing every package registered with Brane together with its exposed functions and parameter types; the Datasets catalog (not pictured) follows the same layout for registered datasets.	26

LIST OF FIGURES

- 3.6 The Evaluation page, comparing compile rate, execution rate, and output match rate across evaluated models, with a per-intent browser for inspecting individual model outputs side by side. 27
- 4.1 The PROV-DM graph produced for one request (solid, top row and agent) and for a cache hit against it (dashed, bottom row). The `Workflow` entity is typed `BraneScriptWorkflow` and the `Result` entity `ExecutionResult` in the exported document; both are shortened here for space. A cache hit does not run a new `TranslationActivity`: its intent is `alternateOf` the one that was already served, and the workflow it reuses `wasDerivedFrom` the original, rather than being generated independently. 37

List of Tables

Introduction

1.1 Motivation

Scientific computing increasingly depends on complex, multi-step computational workflows: data must be fetched, cleaned, analysed, and interpreted, often across heterogeneous compute infrastructure. Workflow Management Systems (WMSs) such as Pegasus (1), Nextflow (2), and Snakemake (3) have become indispensable in fields like genomics, epidemiology, and climate science precisely because they automate resource allocation, fault recovery, and execution ordering. Yet, despite their maturity, these systems impose a heavy cognitive cost on scientists. To express even a straightforward intent—“calculate the average age of patients in this dataset”—a researcher must first understand the WMS’s execution model, write a domain-specific language (DSL) script that explicitly enumerates every computational step, declare data dependencies as a directed acyclic graph (DAG), and then debug execution failures at the infrastructure level rather than the scientific level.

This mismatch between *scientific intent* and *workflow specification* has been recognised as a fundamental barrier to reproducible science. Scientists spend substantial effort debugging YAML configurations and DAG topologies instead of advancing research questions. The cognitive overhead compounds further in distributed settings, where data locality, containerisation, and security policies add additional layers of complexity to what should be a simple computational request.

Recent advances in Large Language Models (LLMs) open a new possibility: a researcher could describe a computational task in plain language, and an intelligent system could automatically translate that natural-language intent into a runnable workflow. This vision, *intent-based orchestration*, has attracted growing interest in the systems and HPC communities. However, existing approaches remain incomplete. Systems that accept natural

1. INTRODUCTION

language either generate generic code that cannot be executed in a target WMS environment, or they lack the formal provenance guarantees required for reproducible science. Conversely, WMSs that provide strong reproducibility have no mechanism for accepting high-level declarative intent.

The gap is thus precise and actionable: no system currently bridges declarative, natural-language intent expression with the rigorous, provenance-tracked execution guarantees of a production WMS.

A second, largely orthogonal problem compounds this gap in practice. Many of the scientific domains that would benefit most from intent-based orchestration, like the one targeting healthcare and genomics, work with data that cannot be freely centralised or shared with an external computing party. An intent-based system for these domains therefore faces a constraint that a general-purpose coding assistant does not: not only the underlying data, but every artefact derived from an interaction with it — the generated workflow, any intermediate or derived results — must remain within the boundary of the organisation that owns the data. A solution to the intent-translation problem above that assumes free data movement, or that must route intents through an external, centrally hosted LLM service, is not viable for these domains regardless of how accurate its translations are.

1.2 Problem Statement

In such distributed scientific computing infrastructures, the party executing a computation need not own the underlying data. The core problem this thesis addresses is how a researcher, working under that constraint, can describe a computational task in natural language and reliably obtain a workflow execution that is both correct and reproducible.

The problem decomposes into four interacting challenges.

Ambiguity of natural language. The same scientific intent can be phrased in many different ways, and different phrasings, or the same phrasing repeated by different users, should ideally produce semantically equivalent workflows. Intents are also frequently underspecified: a request may omit which dataset, parameter, or output format is intended, in which case the system must either recover a reasonable default from context or explicitly ask for clarification, rather than silently guessing. Natural language also uses domain vocabulary (e.g., “heart-disease data”) that does not correspond directly to any identifier in the target system (a specific dataset name or package); the translation system must therefore ground loosely specified terms in the concrete capabilities actually available to it, not merely in the words of the request.

Correctness with respect to a strongly typed, platform-specific DSL. Workflow specification languages of this kind are typically statically typed, with strict, platform-specific rules for invoking computational capabilities (functions, packages, or services) and for connecting their inputs and outputs. Such languages are unlikely to be well represented in the data an LLM was pre-trained on, so a general-purpose model cannot be expected to produce valid programs in them purely from its prior knowledge; it must be additionally grounded in the platform’s actual capabilities, whether through retrieved documentation, fine-tuning on representative examples, or both. Translation errors in this setting fall into two qualitatively different classes that require different detection mechanisms: *overt* errors, where a generated workflow fails to compile or fails at runtime, and *silent* errors, where a workflow executes without error but computes a scientifically incorrect result, which can only be caught by comparing execution behaviour against an expected outcome rather than by static checking alone.

Reproducibility. A scientifically useful translation system must ensure that the same intent, or an evident paraphrase of it, yields the same (or a demonstrably equivalent) workflow and execution trace across repeated invocations, across sessions, and potentially across different users. This requires more than a translator that happens to behave deterministically for a fixed input: because the underlying LLM component is stochastic and may be updated or replaced over time, reproducibility requires an explicit mechanism that relates a given intent to the workflow(s) it has previously produced, so that repeated or equivalent requests do not silently diverge, and so that computation is not repeated unnecessarily for a request that has effectively already been answered.

Operating within a privacy boundary. As established in Section 1.1, in domains such as healthcare and genomics neither the underlying data nor any artefact derived from it — the generated workflow, intermediate results, execution logs — may leave the organisation that owns the data. This constrains not only what the translation system is allowed to output, but where its components are allowed to run: an architecture that routes intents through an externally hosted, centralised LLM service, or that stores any derived artefacts outside the data owner’s boundary, is not a viable solution to the three challenges above, regardless of how accurate or reproducible its translations otherwise are.

Addressing these four challenges together requires not only a translation mechanism from natural language to a target workflow language, but also a retrieval mechanism to ground translation in the platform’s available capabilities, a validation mechanism that distinguishes overt from silent errors, and a mechanism for relating a given intent to workflows it has previously produced, all without exposing data or derived artefacts beyond their

1. INTRODUCTION

owning organisation. How these requirements are met — including the specific mechanisms chosen for grounding, validation, and reuse — is a matter of design and is deferred to Chapter 4.

1.2.1 Scope of This Thesis

The four challenges above are general to any distributed, privacy-sensitive workflow platform with a strongly typed specification language; they do not, by themselves, depend on any particular choice of platform. This thesis studies them concretely through **Brane**, a distributed WMS whose federated execution model dispatches computation to the site holding the relevant data rather than moving the data to the computation, which makes it a natural instantiation of the privacy boundary described above. Brane and its workflow language, BraneScript, are introduced in detail in Chapter 2. From this point on, the thesis is stated concretely in terms of BraneScript — the research questions, contributions, and evaluation all target it directly — while the underlying translation, retrieval, validation, and reuse techniques are not specific to BraneScript and are intended to generalise to other strongly typed workflow DSLs.

1.3 Research Questions

This thesis investigates the following research questions:

- RQ1** To what extent can LLMs translate natural-language scientific intents into syntactically and semantically correct DSL workflows?
- RQ2** How does the quality of retrieval-augmented context and prompt engineering affect translation accuracy, independent of model scale?
- RQ3** How does supervised fine-tuning (SFT) on domain-specific Branescript examples affect translation accuracy and generalisation?
- RQ4** Can semantic caching effectively serve as a reproducibility layer for intent-based orchestration, and what are its fundamental limitations for scientific computing?

1.4 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 provides background on scientific workflow systems, provenance, intent-based orchestration, and LLMs in scientific

computing. Chapter 3 presents a high-level architecture of the proposed system. Chapter 4 describes the design decisions. Chapter 5 details the implementation. Chapter ?? reports the empirical results. Chapter ?? interprets the results in the context of the research questions. Chapter ?? discusses threats to validity. Chapter ?? surveys related work, and Chapter ?? concludes with future directions.

2

Background

This chapter provides the conceptual foundations necessary to understand the remainder of the thesis. We cover scientific workflow systems, data provenance, FAIR principles for computational workflows, intent-based orchestration, and the use of LLMs in scientific computing. We conclude with the selection of a target platform for this work: Brane, the execution platform this thesis targets, and Dynamos, an earlier alternative considered but not adopted.

2.1 Scientific Workflow Management Systems

A *scientific workflow* is a structured composition of computational steps whose execution produces a reproducible scientific result. Workflow Management Systems (WMSs) automate the orchestration of these steps across potentially heterogeneous compute resources, handling scheduling, data transfer, error recovery, and resource allocation transparently from the scientist (1).

2.1.1 Representative Systems

Several WMS platforms have achieved broad adoption in the scientific community.

Pegasus (1) is a DAG-based system developed at USC/ISI that abstracts over heterogeneous grid and cloud resources. It supports four distinct lifecycle phases: workflow generation, workflow planning (site selection and data staging), workflow execution, and monitoring. Pegasus uses HTCondor as its underlying execution engine and emphasises fault tolerance through automatic retry and checkpointing.

Nextflow (2) adopts a dataflow programming model in which independent computational steps (processes) communicate via asynchronous channels. It has become dominant

2.1 Scientific Workflow Management Systems

in bioinformatics due to its native Docker and Singularity container support and its community registry of reusable pipelines (nf-core). Nextflow’s DSL2 syntax is Groovy-based and explicitly describes data movement between processes.

Snakemake (3) follows a *make*-inspired declarative model: scientists define rules that describe how to produce output files from input files, and the system infers the necessary execution order. It is particularly popular in genomics and integrates natively with conda environments for software dependency management.

Galaxy (4) provides a web-based visual interface for workflow construction, lowering the barrier for non-programmer scientists. Workflows are composed graphically and can be shared through public repositories.

Common Workflow Language (CWL) (5) is a vendor-neutral specification rather than an execution engine. It defines a portable, YAML-based format for describing tool invocations and their data connections, with the goal of enabling workflow portability across WMS implementations.

Despite their diversity, all these systems share a common limitation from the perspective of this thesis: they require the scientist to specify workflows explicitly and at the computational level. The transition from a scientific intent to an executable workflow is left entirely to the researcher.

2.1.2 Workflow Lifecycle

A typical WMS workflow follows five phases (1): (1) *specification*, where the scientist describes the computational steps and their dependencies; (2) *planning*, where the WMS maps abstract steps to concrete computational resources; (3) *execution*, where steps run on target infrastructure; (4) *monitoring*, where execution progress and failures are tracked; and (5) *archiving*, where results and provenance are stored for future reference. This thesis focuses on two of these five phases. Its primary focus is the interface before phase 1: the translation of natural language into a formal workflow specification. Its secondary focus is phase 5: the translation system introduced here is required, by the reproducibility and privacy constraints motivated in Section 1.1, to record which intent produced which workflow, so that repeated or paraphrased requests can be related to prior executions rather than re-derived from scratch. Phase 5 is therefore not only a downstream bookkeeping step but a design target of this thesis, and it is the reason the remainder of this chapter treats data provenance (Section 2.2) as a first-class topic rather than as background context alone.

2. BACKGROUND

2.2 Data Provenance

Reproducibility in scientific computing requires not only that results can be recomputed, but that the precise computational history that produced those results is recorded and queryable. This is the domain of *data provenance*, and it has been recognised as a first-class concern for scientific workflows for over a decade: workflow systems are expected not merely to execute a computation but to leave behind a trustworthy record of what was run, on what inputs, and by whom, so that results can later be verified, audited, or reused (6). Provenance therefore serves at least three distinct purposes in a scientific workflow system: it lets a third party *verify* that a published result actually follows from the stated inputs and method; it lets a scientist *debug* a workflow by tracing an unexpected output back to the step or parameter that produced it; and it lets a data owner or auditor *attribute* responsibility for a computation to the specific request, user, and code version that caused it to run — an attribution requirement that becomes central once, as in this thesis, the request itself originates from natural language rather than from a hand-written script.

2.2.1 W3C PROV Data Model

The W3C PROV Data Model (PROV-DM) (7) provides a standard, domain-independent vocabulary for expressing provenance, developed as a W3C Recommendation to allow provenance produced by different, otherwise incompatible systems to be exchanged and reasoned about uniformly. Its core concepts are *entities* (data artefacts, e.g., a dataset or a generated program), *activities* (computational steps that act on or produce entities, e.g., a translation or an execution), and *agents* (the people, organisations, or software processes responsible for those activities). A small set of relationships connect these elements into a provenance graph that captures the causal history of any output: **used** (an activity consumed an entity), **wasGeneratedBy** (an entity was produced by an activity), **wasAssociatedWith** (an activity was carried out by an agent), **wasAttributedTo** (an entity is attributed to an agent), and **wasDerivedFrom** (an entity was transformed, or copied, from another entity without naming the intervening activity explicitly). Because PROV-DM is deliberately generic, it is not tied to any workflow language or execution engine; consequently, essentially every domain-specific provenance format in scientific computing, including the two discussed below, is defined as a specialisation of PROV rather than as an unrelated model.

Figure 2.1 illustrates this core graph structure for a generic scientific computation, using the standard PROV-O visual notation (yellow ellipses for entities, blue rectangles for ac-

tivities, and orange pentagons for agents). A *Computation* activity **used** an input *Dataset* entity and is **wasAssociatedWith** the *Researcher* agent who initiated it; the *Result* entity it produced **wasGeneratedBy** that activity and is, in turn, **wasAttributedTo** the same agent. This is the minimal graph that any of the domain-specific formats discussed in the remainder of this section extend or specialise.

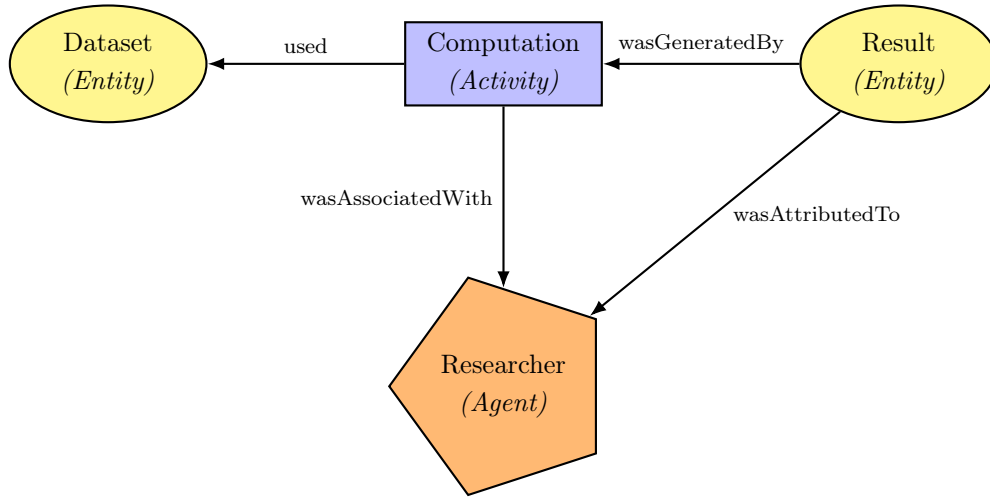


Figure 2.1: The core PROV-DM graph: entities (yellow ellipses), activities (blue rectangles), and agents (orange pentagons) connected by the **used**, **wasGeneratedBy**, **wasAssociatedWith**, and **wasAttributedTo** relations, following standard PROV-O visual notation.

2.2.2 Prospective vs. Retrospective Provenance

Provenance is commonly distinguished along two axes. *Prospective provenance* captures the workflow specification—the plan of what will be executed—and is equivalent to the workflow definition itself. *Retrospective provenance* captures what actually ran, including concrete inputs, parameter values, timestamps, and intermediate results. Full reproducibility requires both: the specification must be re-runnable (prospective), and each execution must be uniquely identified (retrospective). **ProvONE** (8) makes this distinction explicit as a PROV extension purpose-built for scientific workflows: it adds workflow-specific entity and activity subtypes (e.g., distinguishing a *Program*, a static prospective artefact, from an *Execution*, its retrospective instantiation) so that the same PROV vocabulary can describe both the plan and the run without conflating them.

2. BACKGROUND

2.2.3 Workflow Provenance Formats

Several formats extend PROV-DM for workflows. **CWLProv** (9) records the full retrospective execution of a CWL workflow — every step invocation, concrete input and output file, and intermediate value — as a PROV-O graph packaged inside a Research Object (RO) bundle, so that a CWL run can be re-inspected or re-executed independently of the original environment. **RO-Crate** (10) is a more lightweight packaging format based on schema.org that bundles data with its metadata and provenance in a single, self-describing directory, enabling FAIR sharing of computational results without requiring the full PROV-O vocabulary. Both formats, however, are designed around *retrospective* execution provenance: they assume the input to be recorded is already a concrete, machine-executable workflow, and neither has a defined place for the natural-language request that led a human, or an automated translator, to construct that workflow in the first place.

Intent-based systems add a new provenance challenge: not only must the execution be recorded, but the *intent* that triggered the execution must also be captured. This is what we term *intent provenance*: the mapping from a natural-language request to the workflow that was generated and run. Modelling intent within PROV-DM does not require a new top-level concept; an intent can be represented as a specialised *entity* that is **used** by a translation *activity*, which in turn **wasGeneratedBy**-relates to the resulting workflow entity, mirroring how CWLProv and ProvONE specialise PROV’s entities and activities for their own domains. What none of the existing provenance formats address is the semantics of that relationship when the translation activity is not deterministic: because the translation step in this thesis is performed by an LLM, the same intent entity can be **used** by two distinct translation activities that each **wasGeneratedBy**-produce a different workflow entity, so **wasGeneratedBy** alone cannot certify that a request has been answered reproducibly. This gap is one of the motivating design considerations of the semantic cache described in this thesis (Chapter 4): it is precisely the missing mechanism that lets a repeated or paraphrased intent be related, via **wasDerivedFrom**, to a workflow entity that was already generated, rather than triggering an independent, potentially divergent generation event.

2.3 FAIR Principles for Workflows

The FAIR principles—Findable, Accessible, Interoperable, and Reusable—were originally articulated for scientific data, but have since been extended to computational workflows (11).

FAIR workflows require that workflow specifications be discoverable in registries, accessible via stable identifiers, portable across execution environments, and documented with sufficient metadata to enable third-party reuse.

In practice, achieving FAIR compliance for LLM-generated workflows is non-trivial: outputs are generated dynamically, may not be stored in a registry, and may vary across sessions. The semantic cache introduced in this thesis addresses the reusability dimension by ensuring that once a workflow is generated for a given intent, subsequent requests for semantically equivalent intents retrieve the same (or a semantically equivalent) workflow without additional LLM inference.

2.4 Intent-Based Orchestration

Intent-based networking (IBN) is a paradigm that originated in network management, where operators specify *what* a network should do rather than *how* to configure it (12). The system then autonomously translates the high-level intent into low-level configuration. The IBN lifecycle comprises four phases: *expression* (the operator states an intent), *translation* (the system converts the intent to a configuration), *activation* (the configuration is applied), and *assurance* (the system verifies that the intent is satisfied).

2.4.1 Application to Scientific Computing

The IBN paradigm generalises naturally to scientific computing. A scientist expresses an intent (“compute the GC content of this sequence”), the system translates it into a workflow, executes the workflow on the target infrastructure, and verifies that the output is correct. This framing shifts the cognitive burden from workflow engineering to intent expression, which is a task scientists are already equipped to perform.

The MAPE-K (Monitor–Analyse–Plan–Execute with shared Knowledge) control loop from autonomic computing (13) provides a complementary model for adaptive orchestration: the system monitors execution, analyses failures, re-plans the workflow, and re-executes, using a shared knowledge base to inform decisions. This model is relevant when LLM outputs fail to compile or execute, as the system can iterate on the translation rather than surfacing a raw error to the user.

2.4.2 Gaps in the Literature

A systematic review of intent-based orchestration for scientific computing reveals five persistent gaps:

2. BACKGROUND

- G1 — Intent Provenance Model.** No existing system defines a formal model for recording the intent that triggered a workflow execution alongside the standard retrospective execution provenance.
- G2 — Hybrid Intent-to-WMS Bridge.** No system simultaneously supports declarative natural-language intent expression, translation to a target WMS DSL, and WMS-grade reproducibility guarantees. Systems either provide natural-language interfaces without WMS integration, or WMS integration without natural-language interfaces.
- G3 — Intent Reproducibility Definition.** There is no agreed definition of what it means for an intent-based system to be reproducible: does the same intent need to produce the same workflow, or just a semantically equivalent one?
- G4 — Human-in-the-Loop Trust Checkpoints.** Existing proposals for LLM-based workflow generation do not provide principled mechanisms for a scientist to inspect and approve a generated workflow before execution, particularly important in safety-sensitive or privacy-sensitive domains.
- G5 — Cross-Paradigm Benchmarks.** There are no publicly available benchmarks that evaluate intent-based systems against conventional WMS approaches on the same scientific tasks.

This thesis addresses **G2** directly: it implements a hybrid bridge between natural-language intent expression and the Brane WMS via LLM-based BraneScript generation. It provides partial evidence towards **G3** through the semantic cache threshold analysis, and motivates **G1** as future work.

2.5 Large Language Models in Scientific Computing

2.5.1 LLMs for Code and Workflow Generation

LLMs have demonstrated substantial capability in code generation tasks. Models such as OpenAI Codex (14) and its successors have been evaluated on programming benchmarks including HumanEval and MBPP, where they achieve competitive pass@k rates. More recent open-weight models (Qwen, Llama, Mistral families) have closed much of the performance gap with proprietary systems, making local deployment on HPC infrastructure feasible.

2.6 Target Platform Selection: Brane and Dynamos

In the workflow domain, Balis (15) demonstrated that LLMs can generate Parsl-based Python workflows from natural-language descriptions, achieving promising accuracy on domain-specific tasks. Shin et al. demonstrated similar results for bioinformatics pipelines. These works established that LLMs can encode sufficient domain knowledge about workflow patterns to serve as initial translators, but noted that accuracy degrades significantly when the target WMS has a non-standard or less-common DSL—a challenge directly relevant to BraneScript.

2.5.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) (16) augments LLM inference by retrieving relevant context from an external knowledge base and prepending it to the input prompt. In the workflow translation setting, the knowledge base contains package documentation—function signatures, parameter descriptions, and usage examples—and the retrieval system selects the subset of packages most likely relevant to the current intent.

RAG addresses a critical limitation of pure LLM generation: models cannot know the exact API of a custom execution environment such as Brane without having seen it in training data. By retrieving package documentation at inference time, the system grounds generation in the actual available capabilities, reducing hallucinated function calls and incorrect parameter types.

2.5.3 ControlA and Agentic Orchestration

Gueroudji et al. introduced ControlA, a system that uses LLMs as autonomous agents for scientific workflow management, allowing the model to iteratively refine a workflow in response to execution feedback, with the model itself reasoning about how to diagnose and repair failures. This agentic approach differs from the bounded, externally-driven retry mechanism adopted in this thesis (Chapter 4), and represents a complementary direction for future work.

2.6 Target Platform Selection: Brane and Dynamos

2.6.1 Brane and BraneScript

Brane (17) is a distributed, containerised computing framework designed for federated scientific data analysis. It targets scenarios where data cannot leave a site due to privacy or regulatory constraints; (e.g., hospital patient records, genomics data under GDPR), but

2. BACKGROUND

compute results can be returned to a central orchestrator. Brane wraps computational tools in OCI-compatible container images (*packages*), and provides a central registry from which packages can be invoked across participating sites.

Each Brane package exposes a set of typed functions. A function call specifies a package name, a function name, and typed argument values; Brane dispatches the call to a site that holds the relevant data and returns the result to the orchestrator. This model provides strong isolation and auditability: every function invocation is logged, every data transfer is explicit, and every result is versioned.

This federated execution model is the specific reason Brane was selected as the target platform for this thesis. Unlike a workflow written for a generic Python or shell execution environment, a BraneScript workflow can be deployed and executed inside a third-party organisation’s own infrastructure without that organisation ever having to share its raw data with the party that authored the workflow: the workflow travels to the data, not the reverse. This property motivates a corresponding design constraint for the translation system built in this thesis: both the LLM used to generate BraneScript and the semantic cache introduced in Chapter 4 are designed to run locally, within the same trust boundary as the data, so that neither the natural-language intent nor the generated workflow needs to leave the organisation in order to be produced, validated, or reused.

Beyond its federated execution model, two further, more practical properties made Brane a better fit than the alternatives. First, BraneScript’s static typing gives the translation pipeline a machine-checkable oracle: the Brane compiler can reject a malformed generated workflow before any container is started or any data is touched, which is precisely the *overt-error* detection mechanism the problem statement in Section 1.1 requires, and it is a much weaker guarantee that a dynamically typed target language (e.g., a plain Python script) could offer, since there the same mistake would only surface, if at all, as a runtime failure deep into execution. Second, Brane’s package model — typed function signatures published through a central registry — maps directly onto the retrieval problem this thesis needs to solve: a package’s signatures are themselves a natural, structured retrieval corpus for grounding LLM generation (the RAG design described in Chapter 4), which is not true of execution environments whose available capabilities are not declared in any machine-readable form. The alternative considered on exactly these points is discussed next.

BraneScript is the DSL used to express Brane workflows. It is a statically typed, procedural language with Python-inspired syntax. Key constructs include:

2.6 Target Platform Selection: Brane and Dynamos

- **Package imports:** `import package_name`; loads a package’s function signatures into scope.
- **Function calls:** `let result := func(arg := value)`; invokes a package function with named arguments.
- **Data commits:** `commit_result(name, result)`; persists an intermediate result as a named dataset.
- **Control flow:** Standard `if/else` and `for` constructs.
- **Parallel execution:** `parallel [stmt1, stmt2]` executes multiple statements concurrently.

The key challenge for LLM-based generation is that BraneScript is not represented in standard pre-training corpora. Models must infer its syntax from the few examples provided in the system prompt and from the RAG context. Supervised fine-tuning on a curated set of BraneScript examples is intended to internalise the syntactic patterns and package invocation conventions that the base model cannot reliably produce zero-shot; the specific packages and example set used to do so in this thesis are an evaluation testbed and are described in Chapter ??, not a limitation on which packages BraneScript or the translation approach can support.

2.6.2 Alternative Considered: Dynamos

Before settling on Brane, an earlier iteration of this project targeted **Dynamos** (18, 19), a middleware developed at the University of Amsterdam for policy-driven, federated data exchange. Dynamos takes a microservice-based rather than a package/container-based approach: it abstracts common data-exchange patterns into reusable *archetypes*, and uses a MAPE-K control loop (the same autonomic-computing pattern introduced in Section 2.4) to dynamically compose and adapt microservice chains at runtime according to data-sharing agreements between participating institutions. Like Brane, it is built around the premise that data should stay under the control of the institution that owns it, and several exploratory translation experiments were carried out against it early in this project, targeting a small subset of intents.

Dynamos was ultimately not adopted as the platform for this thesis, for two concrete reasons observed during this early exploration. First, the set of available archetypes did not yet cover the breadth of workflow patterns — typed function invocation, control flow,

2. BACKGROUND

and parallel execution — that a general-purpose scientific translation system needs to be able to target, and its microservice composition model was still evolving in ways that would have made a stable target for LLM-based generation difficult to maintain over the course of the project. Second, the underlying Kubernetes-based execution runtime was, at the time, itself unreliable: exploratory test runs intermittently failed or behaved unpredictably at the container-orchestration level, in ways that were not always straightforward to diagnose, which was further evidence that the framework was not yet mature enough to build a production-oriented translation pipeline on top of. Brane’s statically typed package model and comparatively stable runtime made it the more viable choice for the remainder of the thesis, though the underlying motivation — privacy-preserving, federated distribution of scientific computation — is shared between the two systems, and the translation approach developed here does not depend on any feature unique to Brane over Dynamos beyond the availability of a stable, typed target language, compiler, and runtime.

In retrospect, one aspect of Dynamos’s design remains an interesting fit for the intent-based framing pursued in this thesis. An archetype declares *which pattern* of data exchange should be instantiated, leaving the middleware to compose and configure the concrete microservices that realise it; this is arguably closer in spirit to a declarative statement of intent than a BraneScript workflow is, which remains an explicit, imperative sequence of typed function calls that must be fully specified before it can be executed. Had Dynamos been mature enough to serve as the target platform, the natural translation target for an intent-based system built on top of it would plausibly have been archetype selection and parameterisation, rather than full workflow synthesis — a related but distinct problem from the one addressed in this thesis, and one that this work does not attempt to resolve.

3

Overview

This chapter presents the system built to address the problem stated in Chapter 1: translating a natural-language scientific intent into a correct, executable BraneScript workflow. We describe the system from the perspective of a researcher submitting an intent, and present the resulting architecture as it appears from that same vantage point; the design rationale behind these choices is developed in Chapter 4, and their implementation in Chapter 5.

3.1 User Perspective

The researcher uses the system by typing what they want done in plain English: *“I want to analyse all heart-disease data using package Healthcare”*. Nothing about BraneScript’s syntax, a package’s function signatures, or how Brane actually executes the workflow needs to be known beforehand or added to the request.

Entry points. Two entry points expose this interaction, both built around the same cache-lookup, decomposition, retrieval, and generation steps (Section 3.2 and Chapter 5 give the full detail), but aimed at different use cases:

- **Command-line tool** (`src/pipeline.py`). Intended for local development, debugging, and batch experimentation, this entry point is invoked directly on a machine with the model loaded locally, either for a single intent or for a file of intents processed one after another. It runs synchronously to completion, submitting the generated script for execution once ready.
- **Web dashboard** (`frontend/`). This is the entry point intended for interactive day-to-day use. Its *Generate* page is where a researcher types an intent and picks a model;

3. OVERVIEW

the dashboard submits a batch job to the Snellius HPC cluster on a cache miss, polls for completion, and forwards the generated script and any compiler or runtime errors back to the researcher. On failure, it automatically resubmits the request with the error appended as corrective context, up to a fixed number of attempts, without the researcher having to intervene. Generation is only one of the dashboard’s roles, however: Section 3.5 describes the further pages through which a researcher browses past intents, the package/dataset catalog, and model evaluation results.

Outcomes. Regardless of entry point, a submitted intent eventually resolves to one of three outcomes:

1. **Success.** A BraneScript workflow is generated, compiles, executes correctly on the Brane runtime, and its result (and, in the dashboard flow, the workflow itself) is returned to the researcher. The intent-workflow-result triple is also written to the semantic cache, so that a semantically equivalent future intent can be served immediately without invoking the model again.
2. **Corrective retry.** Generation succeeds syntactically but the script fails to compile or fails at runtime against Brane. The compiler or runtime error is fed back into the prompt as corrective context and generation is retried, up to a fixed number of attempts (three, in the current configuration), before the researcher is shown anything.
3. **Failure report.** All retries are exhausted without producing a working workflow. The researcher is shown the last generated script together with the error that caused it to fail, rather than a silent or generic error, so that the intent can be rephrased or the missing information (e.g., a dataset name) supplied explicitly.

Figure 3.3 (Section 3.4) traces the interactive (dashboard) case in detail, showing the full sequence of messages between the researcher, the dashboard, the semantic cache, the generation worker running on Snellius, and the Brane runtime that ultimately compiles and executes the workflow.

3.2 Process Architecture

As both entry points are front doors onto the same underlying architecture, once an intent is submitted it goes through the same sequence of steps. This pipeline is organised around the six logical components that follow:

3.2 Process Architecture

1. The **semantic cache** is consulted first, before any other work is done. It holds previously generated (intent, workflow, result) triples and returns one immediately if the incoming intent is close enough, in embedding space, to one already seen.
2. On a miss, **intent decomposition** breaks the (often compound) natural-language intent into a short list of concrete sub-tasks, so that later stages have a sharper, more explicit description of what needs to be done than the raw sentence alone provides.
3. The **knowledge base and retriever** takes the intent and its sub-tasks and selects the package and dataset documentation most relevant to it, including function signatures, parameter types and usage examples, out of everything registered within Brane.
4. **Prompt construction and LLM generation** combine the retrieved context, a static BraneScript syntax reference, and the original intent into a single prompt, and produce a candidate BraneScript workflow from a (possibly fine-tuned) language model.
5. The **Brane compiler and runtime** are the sole authority on whether the generated workflow is valid: a single **brane workflow run** invocation compiles the script and, if compilation succeeds, executes it against the real, federated data the workflow's functions address.
6. A **feedback loop** connects the last component back to the fourth: a compilation or runtime failure is captured and fed into the next generation attempt as corrective context, for a bounded number of retries, before the researcher sees anything.

Figure 3.1 shows how these six components connect for a single request, independently of which entry point issued it. Physical placement is addressed next, in Section 3.3.

3. OVERVIEW

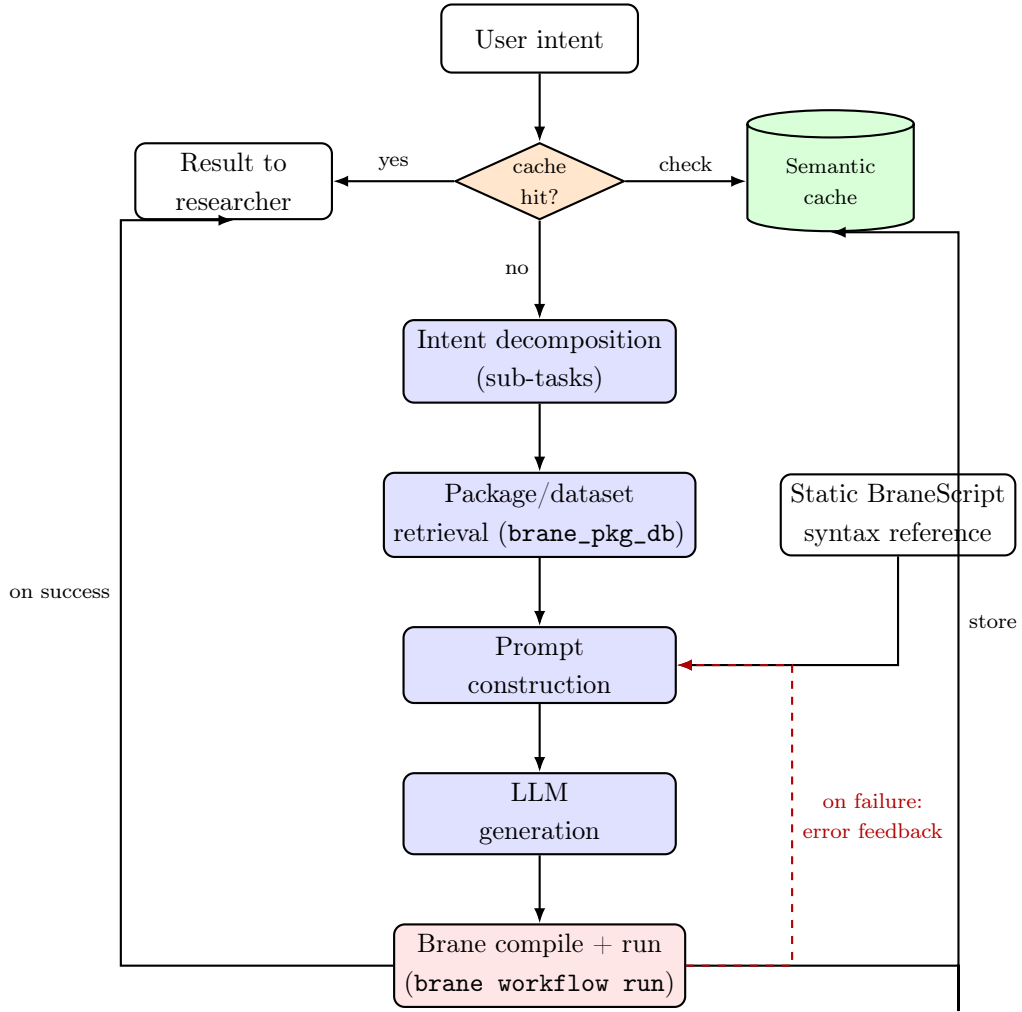


Figure 3.1: Conceptual architecture shared by both entry points. A cache hit returns a previous (intent, workflow, result) triple immediately; on a miss, the intent is first decomposed into a short list of sub-tasks, which sharpen the query passed to the *only* retrieval component in the system, `PkgRetriever`, fetching package and dataset context from a single ChromaDB store (`brane_pkg_db`); the BraneScript syntax reference is not retrieved but injected in full, statically, into every prompt. Brane’s own compiler and runtime, invoked as a single `brane workflow run` command, are the sole authority on whether the generated script is valid; a compilation or runtime failure feeds back into prompt construction as corrective context for a bounded number of retries, and a successful result is written to the semantic cache.

3.3 Infrastructure Architecture

This section describes the physical deployment of the six components introduced in Section 3.2: which machine each one runs on, and how that placement enforces the privacy motivation behind this work (Section 2).

The six logical components fall into three tiers of physical infrastructure. The **dashboard server** is the only tier a researcher’s browser talks to directly; it hosts the frontend and local API process (`server.py`), but holds no model weights and no package or dataset data of its own. On a cache miss, it submits a batch job over SSH to the **Snellius HPC cluster**, where the semantic cache, intent decomposition, package/dataset retrieval, and the fine-tuned LLM all run together as a single SLURM job. These four components are always co-located as the cache gates whether the (expensive) LLM is loaded at all. The command-line entry point collapses the dashboard-server and Snellius-cluster tiers into whichever single machine the researcher runs `pipeline.py` on, but is otherwise architecturally identical.

Downstream of generation sits the **Brane control node**: the dashboard/Snellius path and the command-line path both hand their generated script to it via one `brane workflow run` call. The control node compiles the script and schedules each function’s execution to whichever federated site actually holds the data that function needs and by following Brane’s philosophy, it is not (always) a data-holding site itself. Figure 3.2 shows two such sites, Site A and Site B, as independent examples: each runs its own Brane runtime worker against its own local data, and that runtime, not the control node, is what actually reads the data in place. The control node coordinates and audits every call it schedules, but a site’s data is never routed back through it (unless specifically committed), nor through the Snellius cluster or the dashboard server that produced the script asking for it. This three-tier separation – user interface, LLM-hosting compute, and per-site Brane runtimes – is what lets the LLM-facing part of the system run on whichever hardware is convenient (a laptop, a shared HPC cluster, eventually a cloud-hosted API) without that choice ever placing raw data outside the site that owns it: the LLM never sees a site’s actual data, only the natural-language intent and the package/dataset *metadata* retrieved for it, while the site’s Brane runtime never has to leave its own network to do anything the workflow asks of it.

3. OVERVIEW

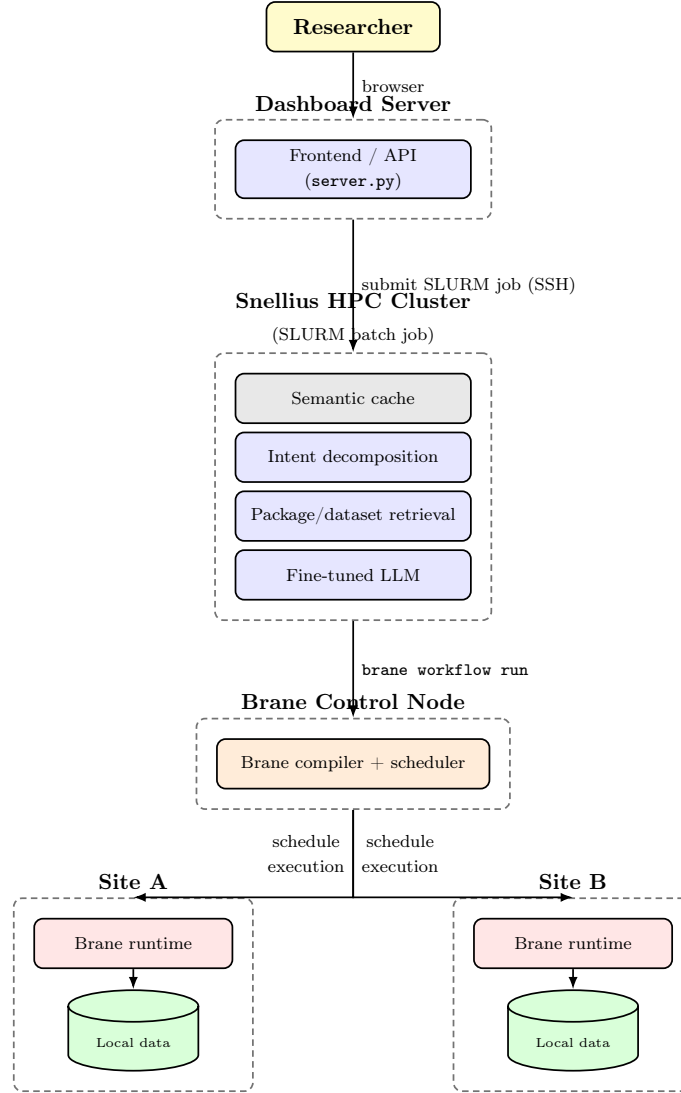


Figure 3.2: Physical deployment of the components shown in Figure 3.1, for the interactive (dashboard) entry point; discussed in the text above. Site A and Site B are shown as two independent examples of federated sites reachable from the same Brane control node.

3.4 Interaction Sequence

Figure 3.1 shows the components involved in a single request but not the timing of the messages between them, in particular the asynchronous submit/poll/retry pattern of the interactive dashboard entry point. Figure 3.3 makes that timing explicit for the case a researcher exercises in day-to-day use: the researcher’s browser, the dashboard backend, the semantic cache, the generation worker running as a batch job on Snellius, and the Brane runtime that ultimately compiles and executes the workflow.

The diagram distinguishes two branches. On a **cache hit** (top frame, steps 1–3), the dashboard looks up the submitted intent against the semantic cache and, on a similarity match, returns the cached script and its previously recorded execution result directly to the researcher; nothing on Snellius or Brane is touched, and no model is loaded. On a **cache miss** (bottom frame, steps 4–14), the dashboard instead submits a SLURM batch job to Snellius and immediately hands the researcher back a request identifier and a “submitted” status (steps 5–6), rather than blocking the HTTP request for however long generation and execution take; the researcher’s browser polls a status endpoint for that identifier until the job completes (step 7). On the Snellius side, the worker performs the process-architecture steps described in Section 3.2 in order – decomposing the intent, retrieving package/dataset context, and generating a candidate script (step 8) – and then submits that script to the Brane runtime with a single **brane workflow run** call (step 9), receiving back an exit code and captured stdout/stderr (step 10). The worker writes this result where the dashboard can find it (step 11); if execution failed and retries remain, the dashboard resubmits the same request with the failure fed back into the prompt as corrective context (step 12), returning to step 5. Once a script executes successfully, the result is written to the semantic cache (step 13) before the final script and outcome are returned to the researcher (step 14).

The command-line entry point (`pipeline.py`) follows the same overall shape – cache lookup, decomposition, retrieval, generation, cache store on success – but differs from the diagram in one respect: it has no equivalent of step 7’s polling, since it runs synchronously to completion on the researcher’s own machine instead of via a remote batch scheduler.

3. OVERVIEW

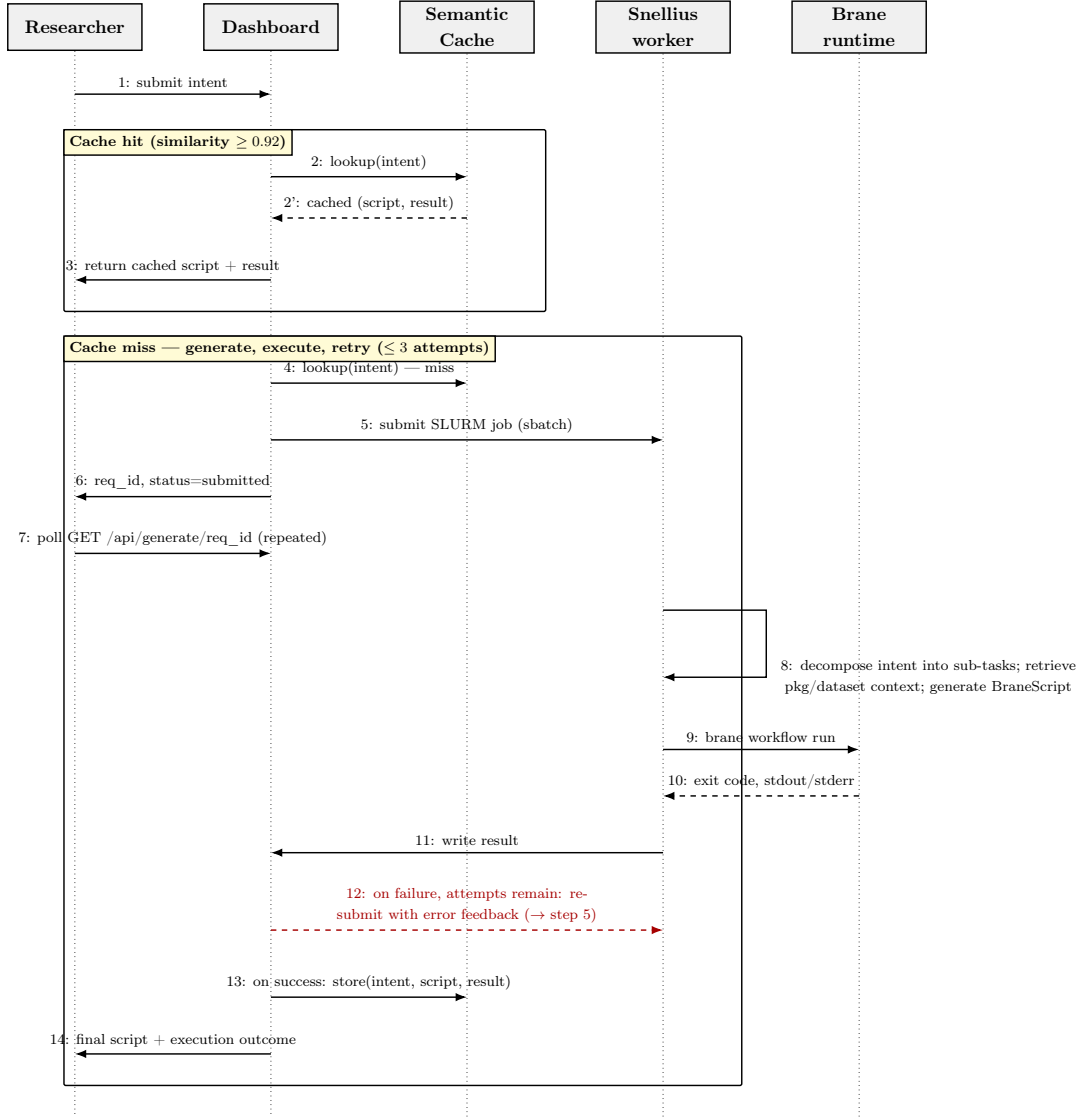


Figure 3.3: Sequence of interactions for the interactive (dashboard) entry point, discussed in the text above.

3.5 Dashboard: Monitoring and Catalog Browsing

The Generate page mentioned in (Section 3.1) is only one of few pages the dashboard exposes. The other five turn it from a pure submission form into a window onto the state of the whole system: what has been tried before, what capabilities are available to build a workflow from, and how the candidate models being fine-tuned actually compare. This matters for the intent-based framing of this thesis specifically because a non-trained researcher who does not have any prior knowledge of BraneScript has no other way to inspect any of this information, as packages, datasets, and model outputs are all hidden behind the natural-language interface. The dashboard’s other pages are:

- **Runs.** A searchable, sortable table of every intent previously submitted through the dashboard, together with the model used, the resulting verdict (pass/fail), the compiler or runtime error type on failure, the total duration, and the number of corrective-retry attempts it took. A stats bar above the table summarises the same information in aggregate (total runs, pass rate, average duration). This is the page a researcher returns to in order to see whether an intent they submitted earlier eventually succeeded, or to gauge, at a glance, how reliably the system handles a given class of intents before trying a new one.
- **Packages and Datasets.** Two browsable catalogs, read directly from the same package specifications (`container.yml`) and dataset manifests (`data.yml`) that back the retriever (Section 3.2): every registered package with its exposed functions and parameter types, and every registered dataset with its access method, README, and example workflows. A researcher can browse what the system is capable of *before* writing an intent, rather than discovering an unsupported capability only after a generation attempt fails.
- **Exec Results.** A table of every real Brane execution produced by the offline evaluation pipeline (Chapter ??), distinct from the interactive Runs table above: each row records the intent, the source training file it came from, the exit code, and the execution time, letting a researcher inspect individual evaluation outcomes rather than only the aggregate metrics reported in the thesis.
- **Evaluation.** A model-comparison table summarising compile rate, execution rate, and output match rate (Chapter 4) across every evaluated model, paired with a per-intent browser that lets a researcher pick a single intent and see, side by side, the BraneScript every model generated for it and whether each one succeeded.

3. OVERVIEW

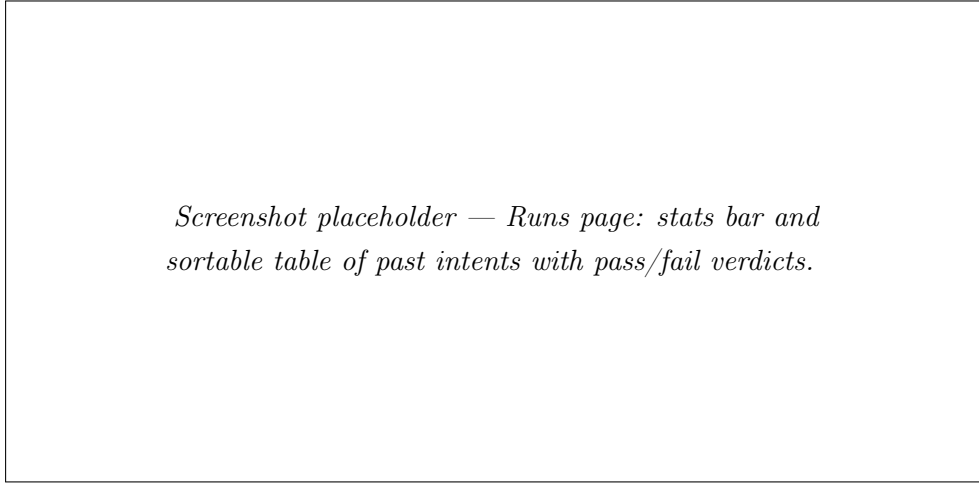


Figure 3.4: The Runs page, showing aggregate statistics and a searchable, sortable history of every intent previously submitted through the dashboard.

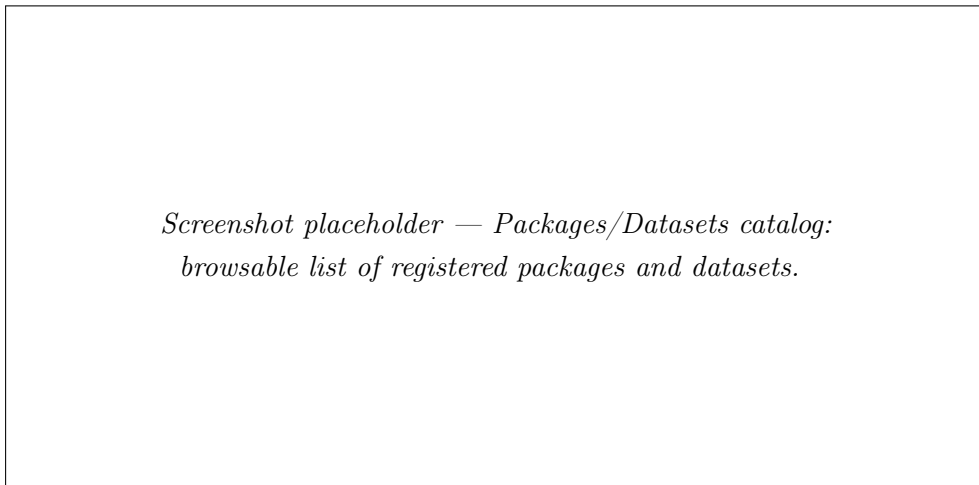


Figure 3.5: The Packages catalog, listing every package registered with Brane together with its exposed functions and parameter types; the Datasets catalog (not pictured) follows the same layout for registered datasets.

3.5 Dashboard: Monitoring and Catalog Browsing

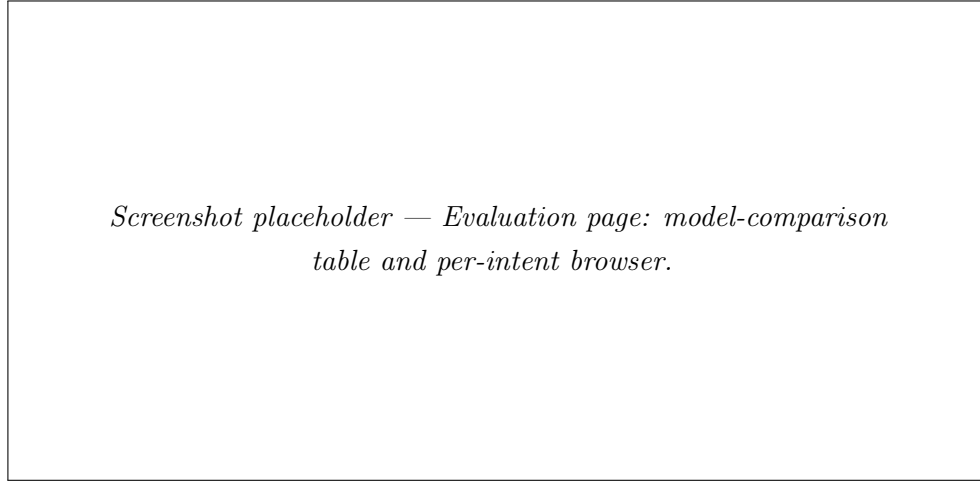


Figure 3.6: The Evaluation page, comparing compile rate, execution rate, and output match rate across evaluated models, with a per-intent browser for inspecting individual model outputs side by side.

Together, the preceding sections describe the same system from increasingly concrete angles: what a researcher submits and sees (Section 3.1), the logical components that turn an intent into an executed workflow (Section 3.2), where each of those components physically runs and why that placement keeps data at its owning site (Section 3.3), how they interact over a single request (Section 3.4), and what a researcher can see of the system’s state beyond a single request (Section 3.5). Chapter 4 discusses the rationale behind the choices summarised in this chapter, and Chapter 5 describes how each component of Figures 3.1 and 3.3 is implemented.

4

Design

This chapter motivates the design decisions behind the architecture presented in Chapter 3. Each decision is presented together with the alternative it was chosen over and the evidence, where available, that justified the choice. It follows the same order in which a request moves through the pipeline: decomposition and retrieval, prompting and typed data representation, dataset and fine-tuning, and finally provenance and caching as cross-cutting concerns.

4.1 Intent Decomposition

Before any retrieval or generation takes place, the pipeline runs the raw intent through a dedicated decomposition step (`IntentDecomposer` in `src/intent_decomposer.py`): a short, separate prompt asks the same already-loaded generation model to break the intent into a numbered list of at most 12 concrete sub-tasks, phrased in plain English rather than code. This step was introduced because retrieving directly against the raw intent conflates two different problems — understanding *what needs to be done* (which BraneScript constructs and package calls the request actually requires) and *how to express it* (concrete BraneScript syntax) — and a single embedding of a multi-part intent tends to average over all of its sub-goals rather than surfacing a chunk for each one. Stating explicitly, in English, that a request needs a package call, a loop over a dataset, and execution on a named node gives the retrieval stage (Section 4.2) several focused queries instead of one broad one; the per-subtask fallback search described there (top-3 chunks per sub-task) only exists because this decomposition happens first, sharpening the query before it is ever embedded.

Two constraints shape what a sub-task is allowed to contain. First, each sub-task must correspond to one primitive BraneScript construct" an import, a function definition, a

conditional, a parallel block, a return statement, and so on, all this deliberately scoped to what the *workflow* has to (and can) express, not what a package function computes internally: the decomposition prompt explicitly forbids sub-tasks such as “compute risk score” or “validate input”, since those describe logic already implemented inside a package rather than something the generated BraneScript itself needs to construct from scratch. This keeps decomposition aligned with the retrieval it feeds: a sub-task describing internal package logic would only ever retrieve documentation about a function’s *behaviour*, not the BraneScript needed to *call* it. Second, each sub-task is capped at 5–15 words and explicitly phrased as “a BraneScript manual search query” rather than a natural-language description, so the same text can be used, unmodified, as the query embedded for retrieval: decomposition and retrieval end up sharing one representation of a sub-task.

One consequence is worth naming explicitly: decomposition never produces code, and its rules ask for none, capping output at a bare numbered list (“No code. No explanations.”). A malformed or partial decomposition therefore cannot itself produce an invalid BraneScript program, as at worst it may weaken the sub-task queries retrieval subsequently runs, degrading which context is retrieved instead of corrupting the model’s actual generation target directly. The step reuses the generation model already loaded in memory rather than loading a second one, so its cost is one extra generation call rather than one extra model; a `-skip-decomposition` flag (`pipeline.py`, `evaluate.py`) disables the step outright, isolating its cost from the rest of the pipeline so its effect on the generated BraneScript can be assessed independently of retrieval and generation.

4.2 Retrieval Design

The sub-tasks produced by decomposition (Section 4.1) are what this stage retrieves against. Gap G2 identified in Chapter 2 requires grounding natural-language translation in the concrete capabilities of the target WMS. This capability surface has two very different shapes, and the design treats them differently rather than folding both into a single retrieval mechanism:

- **BraneScript syntax** — control flow, function declarations, typed `class` declarations, the `parallel` and `on` constructs — is fixed by the language and identical for every intent regardless of which packages it invokes. The full reference (`data/syntax_reference.md`) is about 450 lines (roughly 2,000 tokens), small enough to inject as static text into every generation prompt rather than retrieve. Because the

4. DESIGN

reference does not vary with the intent, a retriever over it could only return a subset of a fixed, already-small document: it would save at most a few hundred tokens per prompt, at the risk of omitting a construct the intent actually needs (e.g., dropping `parallel` on an intent that requires multi-site execution) if the retriever ranks it as irrelevant. Always including the full reference removes that failure mode entirely.

- **Package and dataset context**, by contrast, grows without bound as more packages are registered with Brane, and only a small fraction of it is relevant to any single intent — retrieval buys a real reduction in prompt size here without the correctness risk above. The **package/dataset retriever** (`src/pkg_retriever.py`, backed by the `brane_pkg_db` ChromaDB collection, embedded with `sentence-transformers/all-MiniLM-L6-v2`) indexes function signatures, parameter types, and usage examples for each package registered with Brane, plus metadata about registered datasets, and proceeds in two stages: it first scans the intent for a known package or dataset name (or one of its aliases, see below) and, on a match, retrieves *every* chunk for that package via a metadata filter, guaranteeing complete API coverage rather than a probabilistic top- k selection; only when no name is detected does it fall back to embedding similarity search (top-8 over the intent, top-3 per decomposed subtask), before deduplicating the combined result. In this thesis, this corpus is built from the seven packages used as the evaluation testbed (Chapter ??), but the retriever itself is not specific to those packages.

The retriever’s index is built to grow one package at a time rather than being rebuilt whole. Treating package registration as a full rebuild would have been the simpler alternative: re-embed every package document into a fresh collection any time a package is added, edited, or removed. That alternative was rejected because it ties the cost of adding a single package to the size of the entire catalogue, which works against the premise of the second bullet above — that package context should scale to as many packages as a Brane deployment chooses to register. Instead, a newly registered package is embedded and inserted into the existing collection on its own, leaving every previously indexed package untouched, so the cost of registering one more package does not grow with how many are already registered.

4.2.1 Chunking Granularity

An earlier design indexed each package’s `container.yml` specification as a single chunk (28 chunks total across all packages). This was found to frequently cause the model to select the

wrong function within a package when a package exposed multiple similar functions. The design was revised to *per-function chunking*: every function in every package’s specification becomes its own retrievable chunk (91 chunks total). This small change increases the precision of retrieval at the cost of a larger index, and was adopted because function-level ambiguity was the dominant source of retrieval-induced generation errors observed during development.

4.2.2 Package Aliasing and Hybrid Retrieval

Scientists describe their data using domain vocabulary (“heart disease data”, “epidemic curve”) rather than the internal package names (`healthcare`, `epidemics`) that BraneScript requires, so a name scan against package names alone would miss most intents. Each package directory can define a `keywords.txt` file listing such domain terms; `knowledgeBase.py` indexes it as a document tagged `doc_type="package_keywords"`, and `PkgRetriever` loads these aliases from the collection’s metadata at startup, so a new package’s aliases require only adding the file, not editing retriever code. The name-scan stage described above therefore matches on package/dataset names *or* their aliases before falling back to similarity search. This hybrid was chosen over pure similarity search because similarity search alone occasionally retrieved a semantically related but wrong package (e.g., `statistics` instead of `epidemics` for epidemic-curve intents), whereas exact name or alias matches, when available, are unambiguous.

4.3 Prompt and Feedback Design

The generation prompt combines the original intent, the retrieved package/dataset context, the static BraneScript syntax reference, and a system prompt encoding BraneScript-specific rules (e.g., that empty strings are invalid arguments, that `commit_result` must be called exactly once per named output, and that the model shall only return Branescript valid syntax). These rules were derived iteratively from observed failure modes rather than specified a priori, reflecting the practical reality that the invalid-output space of a fine-grained DSL is best characterised empirically. Both the system prompt and the user-message template are defined once, in `src/prompts.py`: the interactive pipeline, the offline evaluation script, and the dataset-preparation script that produces the fine-tuning corpus (Section 4.5) all call the same `build_user_message` function, so the same inputs always produce byte-identical prompt text regardless of which of the three builds it. This matters because a fine-tuned model learns to associate its target output with the exact textual

4. DESIGN

layout of the training prompts — section headers, their order, how an empty field is rendered. If the dataset-preparation script formatted, say, the package/dataset context block differently from what the pipeline sends at inference, the model would be run on prompts shaped differently from the ones it was fine-tuned on; any resulting drop in output quality would then reflect that formatting mismatch rather than the model’s actual capability.

4.3.1 Failure Modes and Corrective Feedback

When a candidate workflow fails validation, the design reuses the same prompt-construction stage for a bounded number of retries (`MAX_RETRIES = 3` in `pipeline.py`): the failing script and the error that caused it to fail are injected into the next prompt as context, rather than starting generation from scratch. `MAX_RETRIES` is deliberately small and fixed rather than unbounded: if a model has not produced a valid script after three corrective attempts, the failure is treated as a genuine generation failure to be recorded and reported (Chapter ??), rather than something an arbitrarily long retry budget should paper over.

Compiler and runtime error messages from Brane itself are injected verbatim, rather than replaced with a generic “previous attempt failed” message, since Brane already reports which line and construct is at fault. The quality of this feedback matters directly for retry success: a message naming the exact undefined variable or type mismatch gives the model a much stronger signal to correct against than a vague failure would, so the more descriptive Brane’s own error output is, the more likely the next generation attempt is to succeed.

4.4 Typed Data Representation

BraneScript functions that return structured data represent it as tagged class instances (`["ClassName", {field: value, ...}]`) rather than as opaque JSON-encoded strings passed between functions. This design decision was adopted after observing that requiring the model to construct and parse ad hoc JSON strings inside BraneScript led to substantially more hallucinated or malformed intermediate values; typed class instances give the model a smaller, more constrained surface to generate correctly than free-form JSON would.

The alternative – letting the model serialise structured values as backslash-escaped JSON strings (e.g., `let p := "{\\"iterations\\": 100}";`) – was the specific failure pattern the fine-tuned models kept reverting to, likely because it is the natural way to represent structured data in the general-purpose code the base models were originally trained on. The design responds to this on two levels: every structured value a package function accepts or

returns is expressed as a declared `class`, giving a well-typed alternative wherever a JSON string might otherwise be used; and a dedicated check (`detect_json_string_assignment` in `pipeline.py`) inspects each candidate script for this specific antipattern, triggering a retry whose corrective prompt shows the wrong pattern directly next to the correct typed-class equivalent.

The same typed representation is also why the design treats certain Brane values – `Data` references to registered datasets and `IntermediateResult` values produced by package functions – as opaque to BraneScript itself: the model is never asked to inspect or reconstruct their contents from within a script, only to pass them by reference between function calls and, when a result should be persisted, name it once via `commit_result`. Keeping these values opaque removes an entire class of intermediate outputs the model would otherwise have to represent correctly, at the cost of the model being unable to branch on their contents inside a script – a limitation accepted because none of the evaluated intents in this thesis required that capability.

4.5 Dataset Design

The training and retrieval corpus is built around the seven packages described in Chapter 5. An intent/BraneScript pair is included in the fine-tuning corpus only if it is either absent from the recorded batch-execution results (Chapter 5), or present there with a successful exit code; a pair recorded as having failed execution is dropped. Training on every authored example regardless of outcome was considered and rejected, since it would have taught the model, by direct example, that scripts known not to run are acceptable targets to reproduce, precisely at the one stage, training, where such an error is hardest to detect later. The resulting corpus is split into training and validation sets with a fixed 85/15 split and a fixed random seed, so that different fine-tuning runs compared in Chapter ?? are trained and validated on the exact same partition rather than one that varies run to run.

The corpus above is assembled from nine separately authored `.jsonl` files (Chapter 5); a fixed deduplication pass keeps exactly one copy of any near-identical intent that reappears across files, using a fixed file-priority order, so that no single request is over-represented in the resulting train/val split.

A more subtle problem surfaced specifically around the `datetime` package: nearly every one of its functions is, by design, a thin wrapper around the machine’s current wall-clock time, so its output necessarily differs between the moment a training example’s reference output was authored and any later moment the same BraneScript is re-executed. Early

4. DESIGN

evaluation runs surfaced this directly: a generated script that was functionally identical to the reference would still be marked as an output mismatch, since the reference’s stored timestamp could never match a timestamp produced at execution time, regardless of the model’s actual competence, silently deflating the output-match rate for a reason entirely unrelated to translation quality. Since no BraneScript-level fix can make wall-clock time reproducible, the design instead removed the affected examples from the corpus outright, alongside stripping incidentally non-deterministic timestamp fields returned by other packages (e.g. a generation timestamp inside the epidemics package’s report output), so that the fine-tuning and output-match evaluation metrics reported in Chapter ?? only ever compare against genuinely reproducible ground truth.

4.6 Fine-Tuning Design

Several design decisions shape the fine-tuning approach used to produce domain-adapted alternatives to the base LLMs.

Parameter-efficient fine-tuning (QLoRA). Full fine-tuning of a 27B model is infeasible within the available HPC allocation. The design instead uses 4-bit quantisation combined with Low-Rank Adaptation (LoRA), which updates a small number of adapter parameters while keeping the base weights frozen and quantised (exact quantisation and adapter hyperparameters are given in Chapter 5). This makes fine-tuning the largest evaluated model (Qwen3.6-27B) tractable on the available GPU hardware, at the cost of a lower-rank approximation of the full fine-tuning update.

Full-coverage LoRA targeting. The adapter is attached to every linear projection in each transformer block – the four attention projections (query, key, value, output) and the three feed-forward projections (gate, up, down) – rather than to attention alone, which is a common narrower default. BraneScript is a small, rigid DSL whose difficulty for the base model is not primarily about attending to the right tokens in context (what the attention projections adapt) but about reproducing exact syntax, keyword names, and package call signatures token by token, which is governed at least as much by the feed-forward projections; restricting adaptation to attention only was rejected because it would have left the feed-forward pathway – the more likely source of the syntactic and package-signature errors this thesis specifically aims to reduce – frozen. The rank is kept the same across every attention and feed-forward matrix rather than allocated unevenly, and the LoRA scaling factor α is fixed at twice the rank throughout, following the widely used convention that keeps the effective adaptation strength comparable as rank

is changed – avoiding introducing a second hyperparameter that would otherwise need its own justification and tuning budget alongside rank itself.

Training-inference prompt parity. The fine-tuning corpus is not built from raw text but from the exact chat-formatted message list that `prepare_dataset.py` constructs by calling the same `build_user_message` function used at inference time (Section 4.3): a **system** turn carrying the BraneScript rules and syntax reference, a **user** turn carrying the intent and retrieved package/dataset context, and an **assistant** turn holding the reference BraneScript itself, standing in for the reply the model is being trained to produce. The training script itself does no prompt construction of its own; it only applies the tokenizer’s chat template to whatever messages the dataset already contains. This was a deliberate consequence of centralising prompt construction in `src/prompts.py` rather than a separate design decision made inside `train.py`: since fine-tuning teaches the model to associate the reference BraneScript with the exact prompt text surrounding it, routing training data through the identical function used by the pipeline and the evaluation script guarantees that whatever the model learns from is the same distribution of prompts it is later evaluated and used on, rather than a separately maintained approximation of it.

Response-only loss masking. An early version of the training loop computed the language-modelling loss over the entire prompt, including the system prompt and retrieved context, not just the target BraneScript. This diluted the gradient signal for the stop token and caused generation to continue past the intended end of the script at inference time. The design was corrected to mask the loss so that it is computed only over the response tokens (the reference BraneScript), which is the standard approach for instruction-style fine-tuning and directly fixed the over-generation problem observed during development.

Model-agnostic training across scales. The training script reads the base model from an environment variable rather than hard-coding it, so the same training procedure – data, prompt template, LoRA configuration, loss masking – applies unchanged to every model size. This is what makes the three-way comparison in Chapter ?? (Qwen3.5-4B, Qwen3.5-9B, and Qwen3.6-27B, each fine-tuned independently) a precise and reliable controlled comparison of model scale.

Resumable training and checkpoint selection. HPC allocations are time-bounded and jobs can be pre-empted before training completes, so the design checkpoints periodically and, on restart, resumes automatically from the latest checkpoint for that run rather than restarting from the base model (an explicit flag is available to force a clean restart when needed). Checkpoints are evaluated against the held-out validation split at the same

4. DESIGN

interval, and the checkpoint kept at the end of a run is the one with the lowest validation loss, not necessarily the last one written – guarding against the case where additional epochs overfit the training split rather than continuing to improve. Each run also writes its own configuration (base model, LoRA settings, learning rate, dataset sizes) alongside its checkpoints, so that a given adapter’s provenance is not dependent on external notes.

4.7 Intent Provenance

In Section 2.2 we defined *intent provenance* the mapping from a natural-language request to the workflow it produced, alongside the standard retrospective record of how that workflow executed. This type of provenance is identified as a gap no existing provenance format addresses (gap G1). This thesis also aims to contribute to a design that records this mapping for every submitted request. The pipeline, when run with data collection enabled, writes a per-request record containing the same fields: the original intent, the generated BraneScript, the model that produced it and which retry attempt it was, the pass/fail verdict, the specific error type on failure (a JSON-decoding antipattern, a Brane compilation failure, or a runtime failure), and the captured `stdout/stderr/exit` code (`TrainingCollector` in `training_collector.py` for the command-line tool; `save_to_dashboard` in `job_watcher.py` for the dashboard/Snellus path). Every generation event therefore leaves behind an explicit, machine-readable link between the intent entity, the generation activity, and the resulting workflow entity.

These per-request records are serialised as a PROV-DM document by `prov_export.py`, built on the Python `prov` library. For this, we defined a small vocabulary that specialises PROV-DM’s existing entities and activities for this system, in the same way CWLProv and ProvONE specialise them for their own domains (Section 2.2). An intent is a `prov:Entity` used by a `TranslationActivity`, which `wasGeneratedBy`-relates to a `BraneScriptWorkflow` entity and is `wasAssociatedWith` the model that produced it; when the workflow is executed, an `ExecutionActivity` `wasInformedBy` that translation `uses` the workflow entity and `wasGeneratedBy`-relates to an `ExecutionResult` entity carrying the exit code and verdict. Entities and activities are identified by qualified names derived from the originating request’s id (e.g. `bt:workflow-<id>`), so that exporting many records into one document folds shared nodes – the same workflow referenced from two different records – into a single node rather than duplicating it, instead of requiring a separate reconciliation step. Figure 4.1 shows this graph for a single request, together with the cache-hit extension described next.

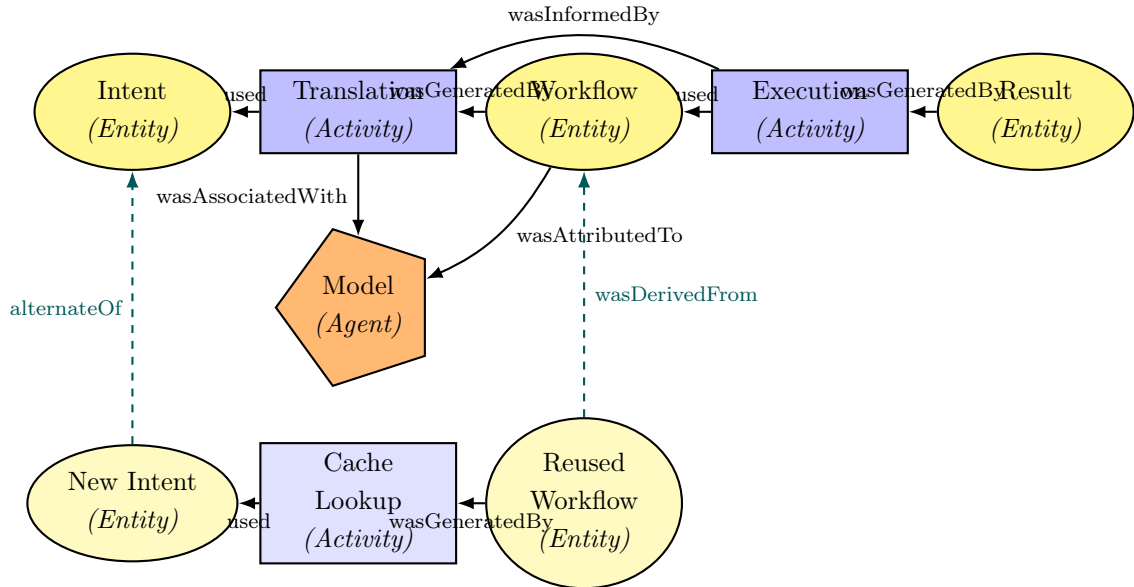


Figure 4.1: The PROV-DM graph produced for one request (solid, top row and agent) and for a cache hit against it (dashed, bottom row). The `Workflow` entity is typed `BraneScriptWorkflow` and the `Result` entity `ExecutionResult` in the exported document; both are shortened here for space. A cache hit does not run a new `TranslationActivity`: its intent is `alternateOf` the one that was already served, and the workflow it reuses `wasDerivedFrom` the original, rather than being generated independently.

4. DESIGN

In fact, a cache hit so far leaves no trace of its own: the pipeline returns the cached script directly without writing a record, so the intent-reproducibility relationship described in Section 2.2 had nothing to serialise. We try to `TrainingCollector.log_cache_hit` close this gap by recording, for every cache hit, the new intent, the id of the run whose cache entry matched it, and the similarity score; the exporter then with this implementation, on a fresh generation now also which run produced the entry it is caching is being recorded, rather than leaving that link empty. With both sides now recorded, the export represents a cache hit as its own small graph: the new intent is `alternateOf` the original one, and the workflow served in response to it `wasDerivedFrom` the workflow entity of the run that originally generated it, rather than from a fresh translation activity, exactly the relationship Section 2.2 identifies as missing from every existing provenance format. The exporter can serialise a single request or merge every run and cache hit under `TRAINING_DATA_DIR` into one document, in PROV-N, PROV-JSON, PROV-XML, or a Graphviz rendering for inspection. What is not being attempted in this thesis is a queryable provenance store: the export is a file produced on demand, not a graph database a researcher can query directly, and stable identifiers are scoped to a single run rather than persisting across the lifetime of a deployment; both are left as future work (Chapter ??).

4.8 Semantic Cache Design

Recording intent provenance, as described above, is not by itself enough to make the system reproducible: the generation activity it records is performed by a non-deterministic LLM, so submitting the same intent again – or the same request paraphrased differently – could still produce a different workflow entity each time, with no relation recorded between the two events. The semantic cache is the mechanism that closes this gap (the intent-reproducibility gap, G3, Section 2.2): the pipeline queries the cache before invoking the LLM at all, and if the same or a paraphrased intent has been served before, the cached `BraneScript` is returned deterministically instead of triggering a fresh, potentially divergent generation event. Only on a cache miss does the pipeline fall back to LLM generation. This is a single fixed lookup-then-generate ordering, not an adaptive or learning mechanism: the cache does not become more or less trusted over time, and no component of the pipeline tracks confidence across requests; a request that can be served from the cache simply never reaches the LLM, which also means it is never re-generated at a fraction of the cost. The cache is consulted by both the command-line tool and the dashboard, but is deliberately excluded from the offline evaluation pipeline used to produce the results in Chapter ??:

a cache hit there would let a previously generated correct answer masquerade as freshly generated output, inflating the reported compile and execution rates without reflecting the quality of the model actually being evaluated.

The cache is implemented as a ChromaDB collection keyed by sentence embeddings of the intent text (using the same `all-MiniLM-L6-v2` model as the retrieval subsystem, for embedding-space consistency), with a configurable similarity threshold (default 0.92) above which a cached entry is considered a hit.

An exact-string-match cache – looking up the intent verbatim rather than by embedding similarity – was considered and rejected as the primary mechanism. It avoids the false-positive risk discussed below, but only ever hits on the literal same sentence being submitted twice, defeating the purpose in a system whose stated premise (Chapter 3) is that a researcher can phrase a request however feels natural to them; two researchers describing the same analysis in different words would never share a cache entry under exact matching.

This is a genuine trade-off, not a settled solution: the same embedding space that enables useful generalisation across paraphrases also maps intents that differ only in a specific parameter (e.g., a numeric threshold or a dataset name) to very high similarity, since sentence embeddings capture propositional meaning rather than exact parameter values. The cache threshold is therefore not a free design parameter to be tuned away, but an explicit precision–recall operating point whose limits are characterised empirically in Chapter ??.

Characterising that operating point requires intents that are genuine paraphrases of an existing cache entry – worded differently but referring to the same underlying request – which the curated dataset does not provide on its own, since it was authored to be varied rather than to contain near-duplicates. `scripts/generate_paraphrases.py` closes this gap by prompting an LLM (Qwen3.5-4B) to reword every training intent several times while keeping its reference BraneScript unchanged, writing the result to `data/training/paraphrases.jsonl`. Each paraphrase is only ever compared against the cache, never merged into the fine-tuning corpus itself (Section 4.5): its purpose is exclusively to sweep the similarity threshold and measure, for each candidate value, how often a paraphrase is matched to any cache entry at all and how often it is matched to the *correct* one, giving the precision–recall figures reported in Section ?? from which the deployed threshold was chosen.

A cache entry is written only once a candidate script has cleared syntax and semantic checking (or, when execution is requested, only once it has also executed successfully), so a script that the pipeline itself rejected can never be served back as a cache hit later.

4. DESIGN

Because the threshold is a similarity cutoff rather than an exact match, the design also has to account for the case where a cached entry turns out to be wrong for reasons outside the pipeline’s own checks (e.g., an underlying package or dataset changes after the script was cached, so a previously correct answer no longer holds). Rather than leave stale entries to accumulate indefinitely, the cache exposes an explicit **invalidate** operation, reachable through the dashboard backend’s `/api/cache/invalidate` endpoint, that removes every entry within the similarity threshold of a given intent, giving a way to correct the cache without needing direct database access. Each entry additionally tracks a hit counter, incremented on every cache hit, giving a simple, low-cost measure of how much reuse a given cached answer is actually getting.

5

Implementation

This chapter details how the design decisions of Chapter 4 are realised in code. It follows the same component breakdown introduced in Chapter 3.

5.1 Pipeline Orchestrator

The orchestrator (`src/pipeline.py`) loops over the generation–validation–execution cycle described in Chapter 3 for at most three attempts per request. Before attempting generation, it queries the semantic cache (Section 5.5) for the intent and returns the cached script and result immediately on a hit; on a miss, it decomposes the intent into sub-tasks via `IntentDecomposer` before retrieval, as described in Chapter 4. A candidate workflow is checked for valid function and argument usage against the retrieved package context before `execute_workflow()` ever submits it to Brane, which compiles and executes it as a single command and returns `stdout`, `stderr`, and an exit code that distinguishes a compilation failure from a runtime failure. On success, the (intent, script, result) triple is written back to the semantic cache and also persisted to disk for later inspection and reuse as training or cache-seeding data. Intents the orchestrator judges too ambiguous to act on are handled separately: rather than attempting generation, it returns a clarifying question to the user.

Setting up a pipeline run once at start-up rather than per-request loads the embedding model, builds the single package retriever shared across the run, and connects to the semantic cache’s ChromaDB collection; if that connection fails, the failure is caught and logged and the run proceeds with caching disabled rather than failing outright, while the BraneScript syntax reference is loaded once as plain text and needs no embedding model at all. A command-line flag allows the cache to be bypassed explicitly for a given invocation.

5. IMPLEMENTATION

5.2 Knowledge Base and Retrieval

`src/knowledgeBase.py` builds and queries the single vector store introduced in Section 4, `brane_pkg_db`, a [ChromaDB](#) collection persisted to disk and accessed through LangChain’s `Chroma` and `HuggingFaceEmbeddings` wrappers, the same LangChain abstractions used throughout the retrieval stack. Package specifications (`container.yml` files) are parsed and exploded into per-function `Document` chunks, each annotated with the owning package name and a short usage hint, and embedded with `all-MiniLM-L6-v2`. Each package’s `keywords.txt` file (Section 5.3) is loaded as its own short, unchunked document tagged with the owning package name; `PkgRetriever` reads these documents back out of the collection at startup and builds an in-memory alias table from them, so the hybrid retrieval strategy described in Chapter 4 never has to guess a package’s domain vocabulary from its function signatures alone. BraneScript syntax is instead loaded `load_system_prompt()` via `data/syntax_reference.md` and formatted directly into the generation system prompt for every request.

As mentioned in (Section 4.2), in order to support the continuous development of the systems, the knowledge base does not need to be rebuilt from scratch when new package or dataset content is added. `knowledgeBase.py` allows the addition of embeddings directly to the existing `brane_pkg_db` collection, so registering a new package only requires embedding that package’s own documents and inserting them, leaving every previously indexed package untouched. This means that adding a new package, or editing an existing package’s `container.yml`, `keywords.txt`, or `README.md`, only costs the time to embed the changed content itself and not the whole catalogue. The current implementation of the pipeline doesn’t detect a stale knowledge base automatically, so re-running the update is still a manual part of the package-authoring workflow rather than one triggered by pipeline startup, but it no longer requires discarding and re-embedding every package already registered. This is the scaling path referred to in Chapter 4: as the number of registered packages grows well beyond the few used in this thesis, the cost of registering one more package stays proportional to that package alone.

5.3 Brane Packages

Seven packages are implemented under `packages/` as the evaluation testbed for this thesis: `healthcare`, `genomics`, `epidemics`, `statistics`, `text_analysis`, `data_masking`, and

datetime. Neither the pipeline nor the retriever is specific to these seven: both operate against whatever typed functions are published in the Brane package registry a given deployment is configured against, and the testbed packages exist to provide a concrete, end-to-end domain on which to train and evaluate the system.

Every package directory follows the same layout: a **container.yml** declaring the package's functions, their typed input parameters and return values, and any custom **class** types it exposes to BraneScript; a single Python source file implementing the actual logic; and a **keywords.txt** listing the domain vocabulary a scientist might use to refer to the package without naming it directly (Section 4.2). Six of the seven packages additionally include a **README.md** documenting, in prose, the package's Brane dispatch conventions and the meaning of each exposed function and class, written for a human maintainer with the goal of extending the package documentation.

Each package follows a common dispatch convention: the container's entry point reads the invoked function name from `sys.argv[1]` (declared in the package's **command.args** specification), rather than relying on environment variables, and routes to the corresponding Python function. Structured return values are serialised as tagged class instances (`["ClassName", {...}]`), consistent with the typed data representation decision in Chapter 4. Every package is of Brane's **ecu** (*executable code unit*) kind, for which Brane itself generates the Docker container image from the **dependencies**, **install**, and **files** fields declared in **container.yml**.

5.4 Fine-Tuning Infrastructure

5.4.1 Training and Evaluation Scripts

`src/fine_tuning/train.py` runs fine-tuning with the base model supplied as a configuration value rather than hard-coded, so the same training code produced the Qwen3.5-4B, Qwen3.5-9B, and Qwen3.6-27B adapters compared in Chapter ?? without any per-model changes. Default hyperparameters are LoRA rank 16 (alpha 32), 3 epochs, an effective batch size of 8, and a learning rate of 2×10^{-4} on a cosine schedule, using 4-bit quantisation through Unsloth where available and a plain Hugging Face/PEFT fallback otherwise; the response-only loss masking described in Chapter 4 is applied at this stage. Training runs on Snellius H100 GPUs and resumes automatically from the latest checkpoint if a job is pre-empted, rather than restarting from the base model, and every run's full hyperparameter set and training progress are logged alongside the resulting adapter so that multiple SFT runs against the same base model remain distinguishable later.

5. IMPLEMENTATION

Evaluation is handled by `evaluate.py`, which computes the three metrics used throughout Chapter ??: *compile rate* (the fraction of generated scripts accepted by the Brane compiler), *execution rate* (the fraction that run to completion with exit code 0), and *output match rate* (the fraction of executed scripts, among those with a stored reference output, whose `stdout` matches the reference exactly). It decomposes each intent into sub-tasks before retrieval by default, using the same decomposition step described in Chapter 4, but can optionally skip this step so that its effect on the reported metrics can be measured directly (Chapter ??). It also deliberately never consults the semantic cache, for the reason given in Chapter 4: a cache hit would let a previously generated correct answer masquerade as freshly generated output, inflating the reported rates without reflecting the model actually being evaluated. Each evaluation run records which training configuration produced the model under test, so that the dashboard (Section 5.7) can distinguish between multiple SFT runs of the same base model without additional bookkeeping.

5.5 Semantic Cache

`src/semantic_cache.py` implements the `SemanticCache` class described in Chapter 4, backed by a ChromaDB collection (`brane_pkg_db/intent_cache`) with a default similarity threshold `DEFAULT_THRESHOLD = 0.92`. Its public interface consists of `lookup(intent)` (cosine similarity search, returning the cached `BraneScript` above threshold or `None`), `store(intent, branescript)`, `invalidate(intent)` (removing stale entries matching an intent), and `stats()/list_entries()` for inspection. The cache is seeded with all 584 unique training intents and their reference `BraneScript` by `scripts/populate_cache.py`, and is consulted before any decomposition or retrieval work is done, by `run_pipeline()` and by `generate_single.py` alike, so that a cache hit avoids model-loading and inference cost entirely, in line with the progressive-trust ordering. `evaluate.py` is the one caller that deliberately never touches the cache, as its whole purpose is to evaluate the actual model performance (Chapter ??).

5.5.1 Cache Evaluation Harness

`scripts/generate_paraphrases.py` produces semantically equivalent rewordings of each training intent (three per intent, targeting 1974 paraphrases total) using an LLM (Qwen3.5-4B by default), via the `sbatch_paraphrase.sh` SLURM wrapper. `scripts/test_cache_lookup.py` then benchmarks cache behaviour by seeding the cache with the original intents and querying it with the paraphrases, reporting: *hit rate* (the fraction of paraphrases that retrieve

any entry above threshold), *correct rate* (the fraction retrieving the exact reference script), *fuzzy correct rate* (the fraction retrieving a script with Jaccard bigram similarity ≥ 0.95 after whitespace/comment normalisation), and *false-positive rate* (the fraction of hits that return the wrong script). Its `-sweep` mode evaluates eight thresholds (0.80–0.98); the implementation queries the cache once per paraphrase at threshold 0 and applies each candidate threshold as a post-hoc filter on the returned similarity scores, reducing the sweep from $8 \times 1974 = 15,792$ ChromaDB queries to 1974.

5.6 Job Watcher

`scripts/remote_execution/local/job_watcher.py` is a small polling daemon, started and supervised by `start.sh` alongside the dashboard, that runs wherever Brane itself is reachable — typically the researcher’s own machine, rather than the Snellius compute node that ran generation. It watches a job directory for BraneScript files written by `generate_single.py`, and for each pending job invokes `subprocess.run(["brane", "workflow", "run", "a", path])` — the same command `execute_workflow()` uses, so a script’s exit code (2 for a compilation failure, 1 for a runtime failure, 0 for success) is always Brane’s own verdict, never a re-implementation of it. This, together with `stdout` and `stderr`, is written back to the job’s result file for the dashboard to pick up and, on failure, feed back into the retry sequence shown in Figure 3.3. Separating job watching from the dashboard process itself keeps the dashboard’s own responsibilities limited to submission, polling, and presentation, and lets Brane execution happen on whichever machine has network access to the target Brane instances, which need not be the machine serving the dashboard.

5.7 Dashboard

The dashboard (`frontend/`, served by `server.py`) exposes six pages: *Runs* (the history of intents submitted through the dashboard, backed by `/api/results` and `/api/stats`, populated from the JSON records `job_watcher.py` writes after each execution); *Packages* and *Datasets* (read-only catalogs served by `/api/packages` and `/api/datasets`, which parse `container.yml` and `data.yml` specifications directly, the same files the knowledge base indexes for retrieval); *Exec Results* (a table over the offline evaluation pipeline’s real Brane execution records, via `/api/execution-results`, kept separate from the Runs table

5. IMPLEMENTATION

since the two are populated by different processes); *Evaluation* (a model-comparison table with a modal showing full training hyperparameters per model, reading the embedded `training_config`, plus a per-intent browser over `/api/eval/runs`); and *Generate* (interactive pipeline invocation). The Generate page submits a SLURM job to Snellius over SSH for a user-supplied intent and selected model, polling a file-based job queue for status (Queued $\rightarrow$ Generating $\rightarrow$ Executing $\rightarrow$ Done) and rendering the resulting BraneScript, execution output, and any errors in a job card. Before submitting a job, `server.py` queries the same semantic cache described in Section 5.5 directly, so that a hit is served immediately without incurring a Snellius round trip at all; this is a convenience lookup against the one cache the system has, not a second, separately deployed instance. On a miss, failed generation attempts are automatically retried up to `MAX_GENERATE_RETRIES = 3` times: on failure, the previous script and its error are written to a context file on Snellius and injected into the next generation attempt’s prompt by `generate_single.py`, mirroring the error-feedback design of the core pipeline (Chapter 4) at the level of the interactive dashboard. Cache inspection endpoints (`/api/cache/stats`, `/api/cache/entries`, `/api/cache/invalidate`) expose the semantic cache’s state directly to the dashboard.

5.8 Infrastructure

A single entry-point script, `start.sh`, starts, restarts, and stops the local services (job watcher and dashboard), writing PIDs to `outputs/logs/` and performing a post-start health check. All run artefacts are consolidated under `outputs/`, with dedicated sub-directories for LoRA adapters and merged weights (`outputs/models/`), evaluation results (`outputs/eval/`), training metrics and plots (`outputs/metrics/`), figures (`outputs/figures/`), pipeline results (`outputs/pipeline/results/`), generated BraneScript (`outputs/branescripts/`), and logs (`outputs/logs/`), replacing an earlier, less structured `output_snellius/` layout.

References

- [1] EWA DEELMAN, KARAN VAHI, GIDEON JUVE, MATS RYNGE, SCOTT CALLAGHAN, PHILIP J. MAECHLING, RAJIV MAYANI, WEIWEI CHEN, RAFAEL FERREIRA DA SILVA, MIRON LIVNY, AND KENT WENGER. **Pegasus, a Workflow Management System for Science Automation**. In *Future Generation Computer Systems*, **46**, pages 17–35, 2015. [1](#), [6](#), [7](#)
- [2] PAOLO DI TOMMASO, MARIA CHATZOU, EVAN W. FLODEN, PABLO PRIETO BARJA, EMILIO PALUMBO, AND CEDRIC NOTREDAME. **Nextflow enables reproducible computational workflows**. *Nature Biotechnology*, **35**(4):316–319, 2017. [1](#), [6](#)
- [3] FELIX MÖLDER, KIM PHILIPP JABLONSKI, BRICE LETCHER, MICHAEL B. HALL, CHRISTOPHER H. TOMKINS-TINCH, VANESSA SOCHAT, JAN FORSTER, SOOHYUN LEE, SVEN O. TWARDZIOK, ALEXANDER KANITZ, ANDREAS WILM, MANUEL HOLTGREWE, SVEN RAHMANN, SVEN NAHNSEN, AND JOHANNES KÖSTER. **Sustainable data analysis with Snakemake**. *F1000Research*, **10**:33, 2021. [1](#), [7](#)
- [4] ENIS AFGAN, DANNON BAKER, BÉRÉNICE BATUT, MARIUS VAN DEN BEEK, DAVE BOUVIER, MARTIN CECH, JOHN CHILTON, DAVE CLEMENTS, NATE CORAOR, BJÖRN A. GRÜNING, AYSAM GUERLER, JENNIFER HILLMAN-JACKSON, SASKIA HILTEMANN, VAHID JALILI, HELENA RASCHE, NICOLA SORANZO, JEREMY GOECKS, JAMES TAYLOR, ANTON NEKRUTENKO, AND DANIEL BLANKENBERG. **The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update**. *Nucleic Acids Research*, **46**(W1):W537–W544, 2018. [7](#)
- [5] MICHAEL R. CRUSOE, SANNE ABELN, ALEXANDRU IOSUP, PETER AMSTUTZ, JOHN CHILTON, NEBOJŠA TIJANIĆ, HERVÉ MÉNAGER, STIAN SOILAND-REYES, BOGDAN

REFERENCES

- GAVRIN, AND CAROLE GOBLE. **Methods Included: Standardizing Computational Reuse and Portability with the Common Workflow Language.** *Communications of the ACM*, **65**(6):54–63, 2022. [7](#)
- [6] SUSAN B. DAVIDSON AND JULIANA FREIRE. **Provenance and Scientific Workflows: Challenges and Opportunities.** In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pages 1345–1350, 2008. [8](#)
- [7] LUC MOREAU, PAOLO MISSIER, KHALID BELHAJJAME, REZA B’FAR, JAMES CHENEY, SAM COPPENS, STEPHEN CRESSWELL, YOLANDA GIL, PAUL GROTH, GRAHAM KLYNE, TIMOTHY LEBO, JIM MCCUSKER, SIMON MILES, JAMES MYERS, SATYA SAHOO, AND CURT TILMES. **PROV-DM: The PROV Data Model.** Technical report, World Wide Web Consortium (W3C), 2013. W3C Recommendation, 30 April 2013. [8](#)
- [8] VÍCTOR CUEVAS-VICENTTÍN, BERTRAM LUDÄSCHER, PAOLO MISSIER, KHALID BELHAJJAME, FERNANDO CHIRIGATI, YAXING WEI, SAUMEN DEY, PARISA KIANMAJD, DAVID KOOP, SHAWN BOWERS, ILKAY ALTINTAS, CHRISTOPHER JONES, MATTHEW B. JONES, LAUREN WALKER, PETER SLAUGHTER, BEN LEINFELDER, AND YANG CAO. **The ProvONE Data Model for Scientific Workflow Provenance.** *DataONE Cyberinfrastructure Working Group, Draft*, 2016. [9](#)
- [9] FARAH ZAIB KHAN, STIAN SOILAND-REYES, RICHARD O. SINNOTT, ANDREW LONIE, CAROLE GOBLE, AND MICHAEL R. CRUSOE. **Sharing interoperable workflow provenance: A review of best practices and their practical application in CWLProv.** *GigaScience*, **8**(11):giz095, 2019. [10](#)
- [10] STIAN SOILAND-REYES, PETER SEFTON, MERCÈ CROSAS, LEYLA JAELE CASTRO, FREDERIK COPPENS, JOSÉ M. FERNÁNDEZ, DANIEL GARIJO, BJÖRN GRÜNING, MARCO LA ROSA, SIMONE LEO, ET AL. **Packaging research artefacts with RO-Crate.** *Data Science*, **5**(2):97–138, 2022. [10](#)
- [11] CAROLE GOBLE, SARAH COHEN-BOULAKIA, STIAN SOILAND-REYES, DANIEL GARIJO, YOLANDA GIL, MICHAEL R. CRUSOE, KRISTIAN PETERS, AND DANIEL SCHOBER. **FAIR Computational Workflows.** *Data Intelligence*, **2**(1–2):108–121, 2020. [10](#)
- [12] IETF IRTF. **Intent-Based Networking—Concepts and Definitions**, 2022. RFC 9315. [11](#)

REFERENCES

- [13] JEFFREY O. KEPHART AND DAVID M. CHESS. **The Vision of Autonomic Computing.** *IEEE Computer*, **36**(1):41–50, 2003. [11](#)
- [14] MARK CHEN, JERRY TWOREK, HEEWOO JUN, QIMING YUAN, HENRIQUE PONDE DE OLIVEIRA PINTO, JARED KAPLAN, HARRI EDWARDS, YURI BURDA, NICHOLAS JOSEPH, GREG BROCKMAN, ET AL. **Evaluating large language models trained on code.** *arXiv preprint arXiv:2107.03374*, 2021. [12](#)
- [15] JAKUB BALIS ET AL. **LLM-Based Workflow Orchestration for Scientific Computing**, 2024. *arXiv preprint*. [13](#)
- [16] PATRICK LEWIS, ETHAN PEREZ, ALEKSANDRA PIKTUS, FABIO PETRONI, VLADIMIR KARPUKHIN, NAMAN GOYAL, HEINRICH KÜTTLER, MIKE LEWIS, WEN-TAU YIH, TIM ROCKTÄSCHEL, SEBASTIAN RIEDEL, AND DOUWE KIELA. **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.** In *Advances in Neural Information Processing Systems*, **33**, pages 9459–9474, 2020. [13](#)
- [17] TIM MULDER, PETER VOS, GIJS SCHOT, AND ALEXANDRU IOSUP. **Brane: A Framework for Distributed Scientific Computing.** <https://github.com/epi-project/brane>, 2022. [13](#)
- [18] JORRIT STUTTERHEIM, ALEANDRO MIFSUD, AND ANA OPRESCU. **DYNAMOS: Dynamic Microservice Composition for Data-Exchange Systems, Lessons Learned.** In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, 2024. [15](#)
- [19] JORRIT STUTTERHEIM. *DYNAMOS: Dynamically Adaptive Microservice-based OS — A Middleware for Data Exchange Systems.* Master’s thesis, University of Amsterdam, 2024. [15](#)

Appendix